

FINAL GRADUATION PROJECT REPORT

Sales Forecasting & Demand Prediction

*An End-to-End Machine Learning System for Retail Demand Forecasting,
MLOps Automation, and Business Decision Support*

Program / Track	AI & Data Science Track — Round 2, Digital Egypt Pioneers Initiative (DEPI)
Project	Project 5: Sales Forecasting and Demand Prediction
Date	July 10, 2026
Version	v1.0 — Final Submission
Primary Dataset	Kaggle — Store Sales: Time Series Forecasting (Corporación Favorita)

Prepared with an end-to-end technical review of the project notebook, deployed API, monitoring pipeline, and dashboard.

Executive Summary

Retail demand is volatile, seasonal, and shaped by dozens of interacting signals — promotions, holidays, paydays, oil prices, and local store dynamics among them. When a retailer misjudges that demand, the consequences show up on both ends of the balance sheet: empty shelves and lost sales when forecasts run low, and markdowns, spoilage, and tied-up working capital when they run high. This project builds, validates, and deploys a machine learning system that forecasts daily unit sales for Corporación Favorita, a large Ecuadorian grocery retailer, at the store–product family–day level, and packages that system into production-style software rather than leaving it as a notebook exercise.

The work draws on the Kaggle “Store Sales – Time Series Forecasting” dataset, which provides five linked tables covering roughly 3.0 million store-family-day sales records across 54 stores from January 2013 to August 2017, together with promotion flags, national/regional/local holiday calendars, daily WTI oil prices, and store-level transaction counts. These sources were profiled, cleaned, merged on their natural keys, and enriched with calendar, holiday, and external-signal features before being fed into a systematic, time-aware modeling process.

Four models were trained and compared under a strictly chronological validation scheme that never allows the model to see the future while learning about the past: a Linear Regression baseline, a randomized-search-tuned Random Forest, a gradient-boosted XGBoost Regressor, and a SARIMA model fitted to the aggregated national daily sales series to test a classical time-series alternative. XGBoost was the clear winner, reaching an R^2 of 0.941 and an RMSE of 331.6 units on a held-out test period of eleven months (August 2016 – August 2017) that the model never touched during training — roughly 2.5× more accurate (by RMSE) than the tuned Random Forest and more than 60% more accurate than the linear baseline.

Beyond model selection, the project implements the full operational loop expected of a production forecasting service: an MLflow-tracked training process, a FastAPI microservice exposing single, batch, and date-range prediction endpoints, a Streamlit business-intelligence dashboard (“PredictAI”) for non-technical stakeholders, a statistical drift-detection module that compares live inference data against the training distribution feature-by-feature, and a retraining pipeline that automatically evaluates whether a freshly trained model should replace the one in production. Applied to this project's own train/test split, the monitoring layer correctly identified that oil prices and promotion intensity have drifted meaningfully between the training and test periods, while confirming that model performance itself had not yet degraded — exactly the kind of early-warning signal a retailer would want before performance actually erodes.

The business case is direct: at a 94% R^2 and a mean absolute error of roughly 96 units against an average of 358 units per store-family-day, the model materially sharpens replenishment, staffing, and promotional-timing decisions, particularly given the confirmed 7.2× sales lift under promotion and the pronounced weekend and December seasonality uncovered during exploratory analysis. The remainder of this report walks through that entire journey — problem framing, data, preprocessing, exploratory and time-series analysis, feature engineering, modeling, validation, MLOps, and business translation — in the technical depth expected of a final graduation deliverable, while being candid about the assumptions, limitations, and improvement opportunities the current implementation leaves on the table.

Table of Contents

Executive Summary.....	2
1. Introduction and Problem Definition.....	3
2. Dataset Description	5
3. Data Collection, Cleaning, and Preprocessing	7
4. Exploratory Data Analysis	9
5. Advanced Time-Series Analysis	17
6. Feature Engineering.....	20
7. Model Development.....	22
8. Training and Validation Strategy	24
9. Evaluation Metrics.....	27
10. Model Comparison	28
11. Final Model Selection	33
12. MLOps, Experiment Tracking, and Monitoring	34
13. Business Impact.....	37
14. Challenges and Limitations	38
15. Future Improvements	40
16. Conclusion.....	41
17. References	42
18. Appendix.....	43

1. Introduction and Problem Definition

1.1 The Forecasting Challenge

Retail demand forecasting asks a deceptively simple question — how much of each product will sell, at each location, on each future day — and answers it under conditions that make the question genuinely hard. Demand is intermittent (roughly three in ten store-product-day combinations in this dataset sell nothing at all), heavily right-skewed (a small number of high-volume rows dominate total revenue), seasonal at multiple frequencies (weekly, monthly, and yearly), and reactive to external shocks such as promotions, holidays, paydays, and macroeconomic conditions. A model that only captures the average pattern will systematically misforecast the events that matter most operationally: the promotional spike, the holiday lull, the December surge.

1.2 Why Demand Prediction Matters in Retail

Sales forecasts are the input to nearly every downstream operational decision a retailer makes: how much stock to order and when, how many staff to schedule on the floor and in the warehouse, when to time a promotional push, and how much shelf space to allocate to a category. Because these decisions are made in

advance of the demand they respond to, their quality is bounded by the quality of the forecast that informs them.

1.3 Operational Consequences of Poor Forecasting

- Stockouts: under-forecasting demand leads to empty shelves, lost sales, and — for a repeat grocery shopper — potential defection to a competitor.
- Overstocking and waste: over-forecasting ties up working capital in inventory and, for perishable categories such as PRODUCE, DAIRY, or MEATS (three of the top revenue-driving families identified in this project's own analysis), directly translates into spoilage and markdowns.
- Misallocated labor: staffing built around an inaccurate forecast either strands payroll hours on a quiet day or under-staffs a store during a genuine promotional or holiday surge.
- Mistimed promotions: without a reliable read on baseline demand, it is difficult to size a promotion's incremental lift or to plan the replenishment a successful promotion will require — a real risk given that this project measured a 7.2× average sales lift under active promotion.

1.4 Business Scope of the Project

The project scope covers the full lifecycle of a forecasting capability rather than a single modeling notebook: data acquisition and cleaning, exploratory and time-series analysis, feature engineering, model development and selection under time-aware validation, and — distinctively for a graduation project — a working MLOps layer consisting of a REST API (FastAPI), a business dashboard (Streamlit), experiment tracking (MLflow), automated drift detection, and a retraining pipeline. The target variable throughout is unit sales (the `sales` column of the training data) at the store × product-family × calendar-day grain.

1.5 What the Model Is Expected to Support

- Short-horizon operational forecasts (days to a few months ahead) for a specific store and product family, callable on demand through the `/predict` and `/predict/range` API endpoints.
- Bulk forecasting for planning cycles, through the `/predict/batch` endpoint (up to 1,000 store-family-date combinations per call).
- Store- and family-level comparative analysis for merchandising and inventory planning, through the Streamlit dashboard's Business Analytics page.
- Ongoing performance assurance, through the monitoring and retraining modules described in Section 12.

2. Dataset Description

2.1 Source and Real-World Context

The project uses the Kaggle competition dataset “Store Sales – Time Series Forecasting” (<https://www.kaggle.com/competitions/store-sales-time-series-forecasting>), built from the real operations of Corporación Favorita, a large Ecuadorian retail conglomerate. For context beyond what is present in the raw files themselves — flagged here explicitly as background rather than a notebook-derived finding — Favorita traces back to a supermarket founded in 1957 and today operates dozens of store formats across Ecuador, Peru, Paraguay, and Panama; the dataset itself covers 54 stores across 22 cities in 16 Ecuadorian states. Two country-level facts make this an unusually rich forecasting dataset rather than a purely synthetic exercise: Ecuador's economy is meaningfully oil-dependent, which motivates including a daily oil-price series as a feature, and public-sector wages are paid bi-weekly (on the 15th and the last day of the month), a pattern known to move grocery demand around paydays. Ecuador was also struck by a magnitude-7.8 earthquake on April 16, 2016, which measurably disrupted supermarket sales for several weeks afterward and falls inside this project's training window — a real external shock that the current feature set does not explicitly flag (see Section 14).

2.2 Why This Dataset Is Appropriate for the Task

- Scale and grain: nearly 3.0 million labeled store-family-day observations provide enough volume to train and time-validate a gradient-boosted model without overfitting.
- Genuine multi-source complexity: five linked tables (sales, store metadata, oil prices, holidays, transactions) mirror the kind of heterogeneous data integration a real retail data science team would face, rather than a single flat file.
- Real seasonality and shocks: multi-year daily coverage captures weekly, monthly, and annual seasonality, an oil-price cycle, and a real natural disaster — all of which this project's own EDA and stationarity testing confirm are present in the data (Sections 4–5).
- A genuine regression target with business meaning: `sales` is directly interpretable as unit sales, so error metrics (MAE, RMSE) translate immediately into operational language (Section 9).

2.3 Files Used and What Each Contributes

File	Rows × Cols	Role in the Project
train.csv	3,000,888 × 6	Core fact table: date, store_nbr, family, sales (target), onpromotion. No missing values or duplicates.
stores.csv	54 × 5	Store dimension: city, state, type (A–E), cluster. Merged on store_nbr.
oil.csv	1,218 × 2	Daily WTI oil price (dcoilwtico). 43 missing values (≈ 3.5%); merged on date.
holidays_events.csv	350 × 6	National/regional/local holiday and event calendar, including transferred-holiday flags. Merged on date (+ state/city for regional/local).
transactions.csv	83,488 × 3	Daily store-level transaction counts, a proxy for footfall. Merged on (store_nbr, date).

Table 1

2.4 Target Variable and Predictors

The target variable is `sales`: the unit sales for a given product family, at a given store, on a given calendar day. The final model consumes 20 input features assembled from all five source files — 16 numeric and 4 categorical (one-hot encoded) — spanning four conceptual groups: calendar/time features (year, month, day, day-of-week, week-of-year, weekend flag), promotional and external signals (onpromotion count, oil price, transaction count), holiday flags (national, regional, local, event, and a combined indicator), and store attributes (store number, cluster, city, state, store type, product family). Section 6 documents the complete feature engineering process, including a number of engineered signals — lag features, rolling means, cyclical encodings — that were explored but ultimately not carried into the final feature set.

3. Data Collection, Cleaning, and Preprocessing

3.1 Missing Value Analysis

A per-column null check was run on all five source tables before any joins were performed. `train.csv`, `stores.csv`, `holidays_events.csv`, and `transactions.csv` contained zero missing values across every column. `oil.csv` was the only table with missingness: 43 of 1,218 daily oil-price observations ($\approx 3.5\%$) were null, corresponding to non-trading days (weekends and market holidays) for which no WTI quote exists.

◆ Handling decision

Oil prices were sorted chronologically and filled with a forward-fill followed by a backward-fill (`ffill().bfill()`). Forward-fill is the natural choice for a slow-moving macro series — it carries the last known market price forward through non-trading gaps — and the trailing backward-fill only covers the rare edge case of missing values at the very start of the series. After this step, `oil.dcoilwtico` had zero remaining nulls.

3.2 Duplicate Checking

Full-row duplicate checks (`.duplicated().sum()`) returned zero for all five tables. A second, key-based duplicate check was then run using each table's natural business key: (date, store_nbr, family) for train, store_nbr for stores, date for oil and transactions, and date for holidays_events. All keys were clean except one: holidays_events showed 38 duplicated dates. This is not a data quality defect — Ecuador's holiday calendar legitimately allows multiple holiday or event entries to share the same date (for example, a national holiday coinciding with a regional or local observance), so the raw table is intentionally long-form. The feature engineering pipeline (Section 3.5) explicitly accounts for this by aggregating holiday flags rather than assuming a one-row-per-date structure.

3.3 Date Handling and Range Checks

All five tables' date columns were parsed with `parse_dates` at load time. Range checks confirmed: `train` spans 2013-01-01 to 2017-08-15; `oil` spans 2013-01-01 to 2017-08-31 (extending slightly past the training window); `transactions` spans the identical range as `train`; and `holidays_events` spans 2012-03-02 to 2017-12-26 — both earlier and later than the training window, which is expected for a general holiday calendar and is handled gracefully by the merge logic (non-matching dates simply contribute no holiday flag).

3.4 Data Type Corrections

Categorical dimension columns on the stores table (`type`, `city`, `state`, `cluster`) were cast to pandas `category` dtype during exploration, and corresponding integer codes (`city_code`, `state_code`, `type_code`, `cluster_code`) were derived for potential use in models sensitive to string columns. In the production feature pipeline, however, the raw string columns are instead passed through a `OneHotEncoder` inside a scikit-learn `ColumnTransformer` (Section 6.6 explains why the manually label-encoded versions were set aside in favor of one-hot encoding).

3.5 Merge Logic Between Tables

[Code block: preprocessing pipeline] The main feature table is assembled with four sequential left joins anchored on `train`, which guarantees the row count of the fact table is preserved end-to-end:

1. `train` → stores on `store_nbr` (adds city, state, type, cluster).
2. → transactions on (`store_nbr`, `date`) (adds daily transaction count; unmatched rows — store/day pairs the transactions table doesn't cover — are filled with 0 after the join).
3. → oil on date (adds `dcoilwtico`; forward/backward-filled again after the join to close any residual gaps introduced by the join itself).
4. → three separate holiday lookups — national holidays on date, regional holidays on (`date`, `state`), and local holidays on (`date`, `city`) — plus a fourth join for events on date. Each of these four lookups is built by first filtering `holidays_events` to `transferred == False` (excluding the 12 holidays that were officially moved to a different calendar date, since a transferred day behaves like an ordinary business day) and then splitting the remaining rows by `type` (Holiday vs. Event) and `locale` (National / Regional / Local).

After all joins, missing holiday flags (rows with no match in any of the four lookups) are filled with 0, and a combined `is_holiday` indicator is derived as the logical OR of the national, regional, and local flags. The resulting merged table has exactly 3,000,888 rows — identical to the original `train.csv` — confirming that none of the four left joins introduced row duplication, and 59 total columns once all raw, derived, and holiday-flag fields are included.

3.6 Handling of Oil, Transactions, and Holidays — Summary

Signal	Missingness Handled	Join Key	Post-Join Fill
Oil price	43 native nulls (ffill/bfill)	date	ffill/bfill again (closes gaps from non-overlapping calendars)
Transactions	None natively; some (store, date) pairs absent from transactions.csv	store_nbr + date	fillna(0)
Holidays	N/A (categorical flags derived)	date / date+state / date+city	fillna(0) for each flag, then OR-combined into <code>is_holiday</code>

Table 2

3.7 Final Cleaning Pass

A final pass converts categorical columns back to plain objects, replaces any residual infinite values (which can arise from percentage-change style features) with NaN, fills remaining numeric NaNs with 0 and remaining object/text NaNs with the literal string “Unknown,” and re-validates that the dataset is clean. The validated result: 0 missing values, 0 duplicate (date, store_nbr, family) keys, and a final shape of 3,000,888 rows × 59 columns, exported to `clean_store_sales.csv` as the Milestone 1 “cleaned dataset” deliverable.

4. Exploratory Data Analysis

With the merged, cleaned dataset in hand, the notebook runs a broad exploratory pass across time, promotions, external signals, and store/product dimensions. This section walks through the strongest, most decision-relevant patterns, each grounded directly in the notebook's own computed values.

4.1 Overall Distribution of Sales

The `sales` column is heavily right-skewed: mean 357.78 units against a median of only 11 units, a standard deviation of 1,102.0, and a skewness coefficient of 7.36. Fully 31.3% of all store-family-day rows record exactly zero sales — an intermittent-demand pattern typical of grocery retail, where any single product family is simply not purchased at a given store on a given day far more often than it is. A standard IQR rule flags 447,105 rows ($\approx 14.9\%$) as statistical outliers; given the confirmed skew, the great majority of these are genuine high-volume selling days rather than data errors, so no rows were removed on this basis — a sound choice, since deleting them would bias the model away from exactly the high-demand days that matter most operationally.

4.2 Weekly Seasonality

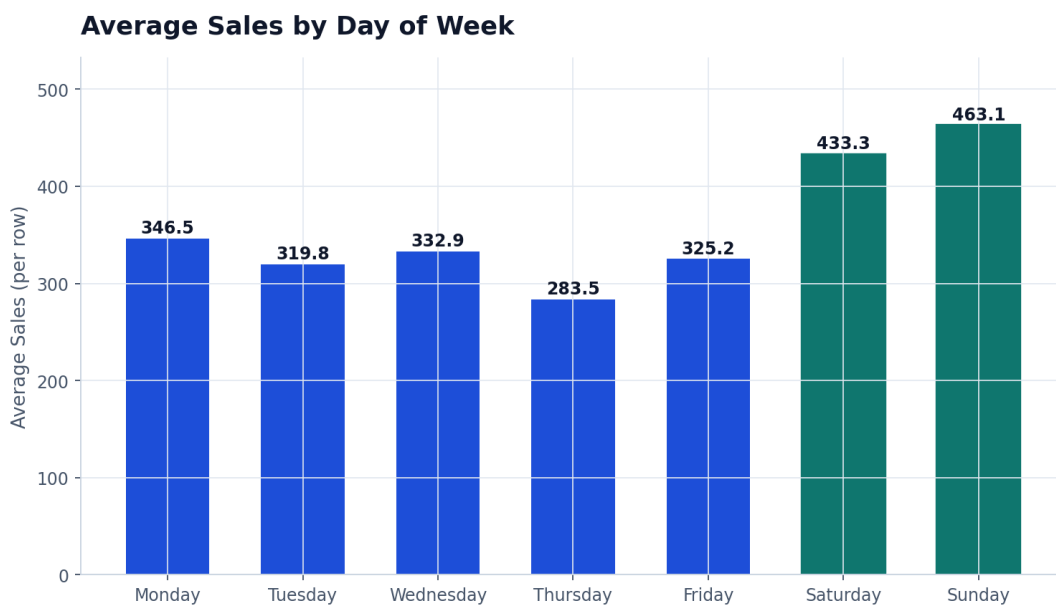


Figure 1. Average sales by day of week. Weekend sales (Saturday, Sunday) are visibly higher than any weekday.

Average sales rise steadily into the weekend: Sunday is the strongest day at 463.1 units, Saturday second at 433.3, while Thursday is the quietest weekday at 283.5. The gap between the strongest and weakest day ($\approx 63\%$) confirms a real, decision-relevant weekly cycle — informing both staffing rhythm and the choice, discussed in Section 5, to give the SARIMA model a 7-day seasonal period.

4.3 Monthly and Holiday-Season Patterns

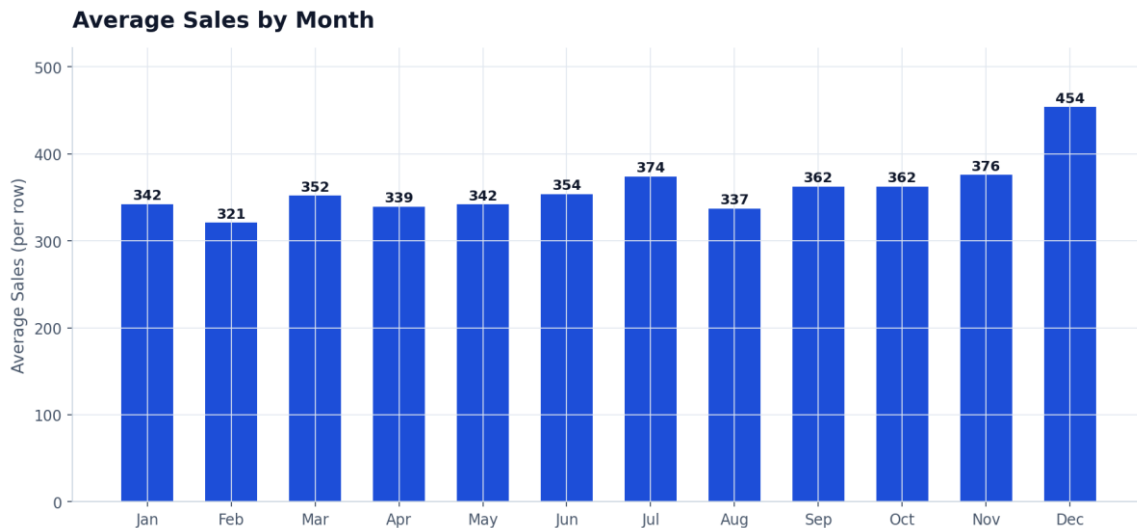


Figure 2. Average sales by calendar month. December stands out as the strongest month of the year.

December is the clear seasonal peak at 453.7 average units, roughly 33% above the year's quietest month (February, 320.9) — an expected holiday-shopping effect that any inventory or staffing plan needs to anticipate well in advance. November (375.7) and July (374.2) are secondary peaks.

4.4 Holiday and Event Effects

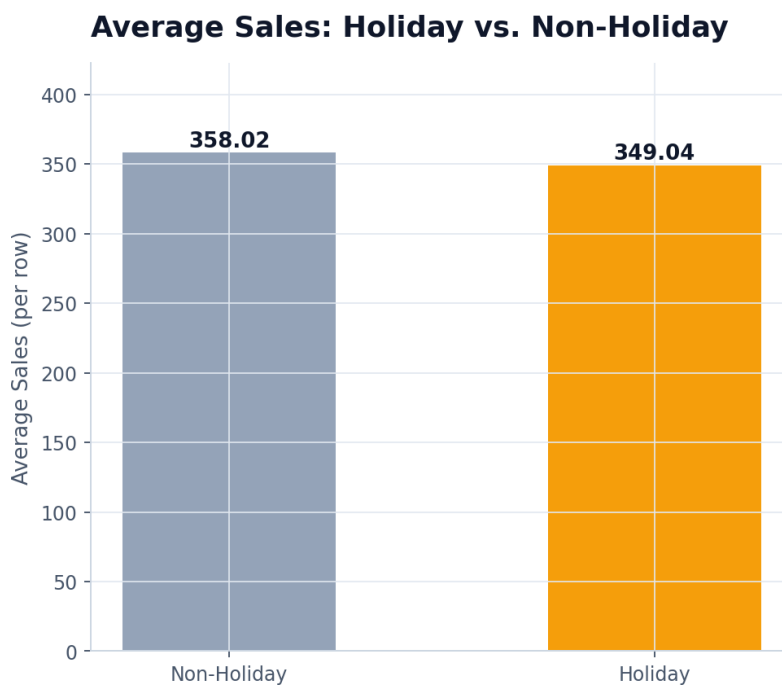


Figure 3. Average sales on holidays vs. non-holidays.

Interestingly, the notebook's own numbers show average sales are essentially flat — and if anything, slightly lower — on flagged holiday dates (349.0) compared with ordinary days (358.0), a roughly 2.5% difference that is small relative to the $\pm 1,102$ -unit standard deviation of sales. This is a genuinely useful, slightly counter-intuitive finding worth stating plainly rather than smoothing over: it likely reflects that the `is_holiday` flag blends many different holiday types (some of which reduce footfall, e.g., holidays associated with store closures or reduced hours, while others increase it) into a single binary indicator, diluting any single directional effect. By contrast, `is_event` dates (concerts, sporting events, and similar occasions in the

Ecuadorian calendar that are not full holidays) show a clearer positive lift: 425.0 average units vs. 355.5 on non-event days, a $\approx 19.6\%$ increase. This suggests that, if pursued further, disaggregating the holiday flag by holiday type (rather than one merged indicator) could sharpen the model — noted again as a future improvement in Section 15.

4.5 Promotion Effects

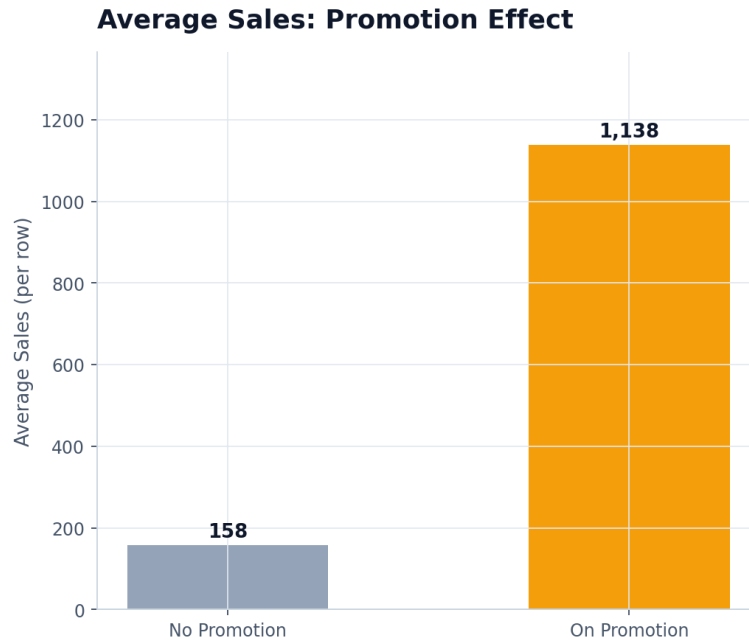


Figure 4. Average sales with vs. without an active promotion on the row.

Promotions have, by a wide margin, the single strongest effect measured anywhere in this EDA: average sales jump from 158.2 units with no promotion to 1,137.7 units when `onpromotion > 0` — a 7.2× multiplier. The Pearson correlation between `sales` and the raw `onpromotion` count is 0.428, the strongest of any numeric predictor tested against sales. This single finding underpins much of the business case in Section 13: accurate promotional-lift forecasting is plausibly the highest-leverage use of this model.

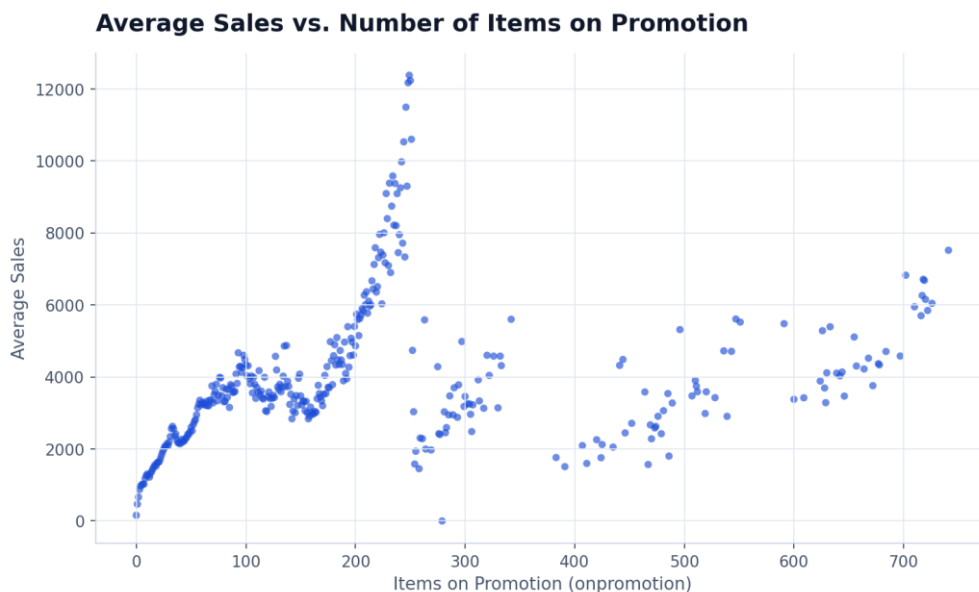


Figure 5. Average sales as a function of the number of items on promotion — a clear positive, mildly saturating relationship.

4.6 Transactions and Oil Price

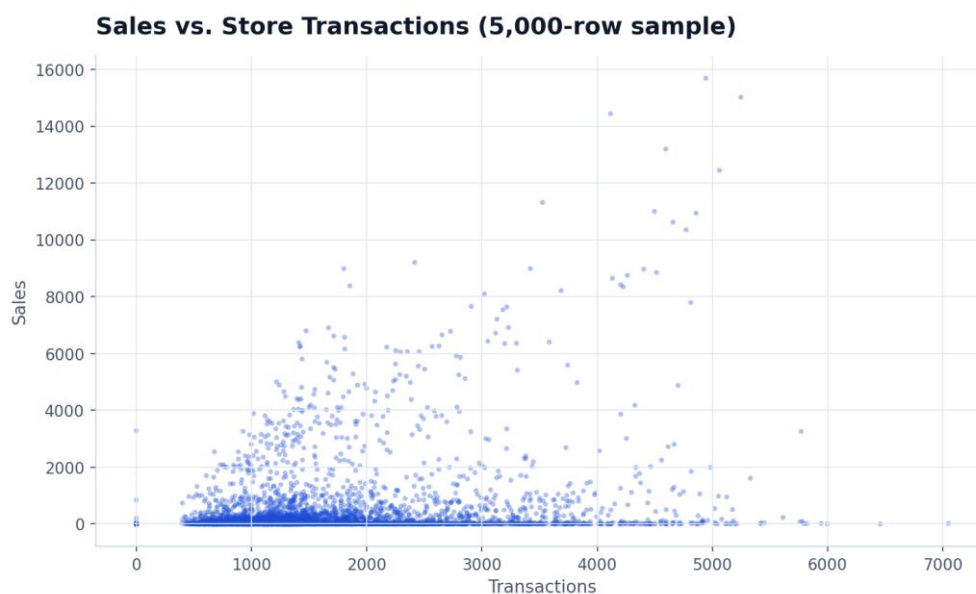


Figure 6. Sales vs. store-level transactions (5,000-row sample). A positive but noisy relationship — transactions capture footfall, not basket composition.

Transactions correlate with sales at 0.233 — positive but noticeably weaker than the promotion relationship, since transaction counts describe store-level footfall while `sales` is measured at the individual product-family level within that same store-day. Oil price shows a very weak negative correlation with sales (-0.075).

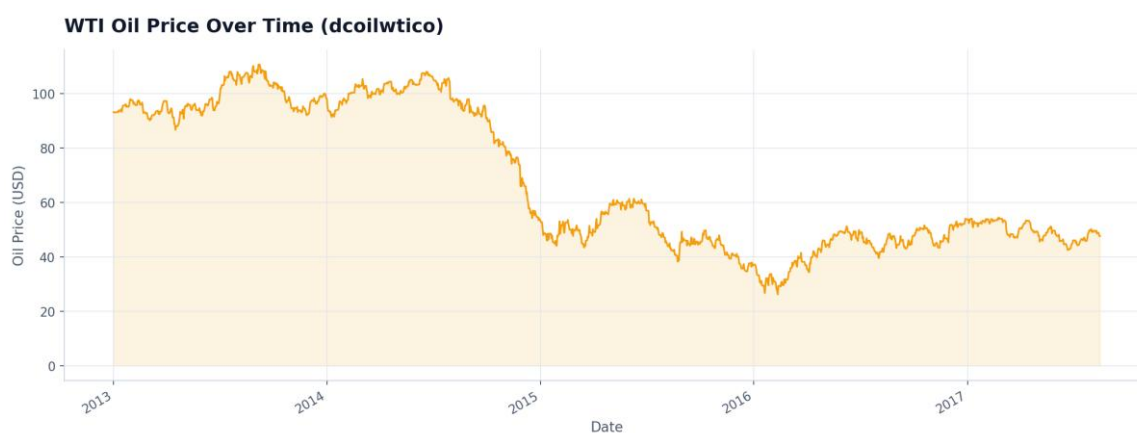


Figure 7. WTI oil price over the training window, including the well-known 2014–2016 price collapse.

The oil-price chart itself is a useful sanity check independent of its weak direct correlation with sales: it clearly reproduces the real 2014–2016 oil-price collapse (from roughly \$100+ down into the \$30–\$50 range), which — as flagged in the monitoring analysis of Section 12 — is also the dominant source of covariate drift between the model's training and test periods, since the chronological split places most of the low-oil-price period in the test set.

4.7 Store, Cluster, and Product-Family Insights

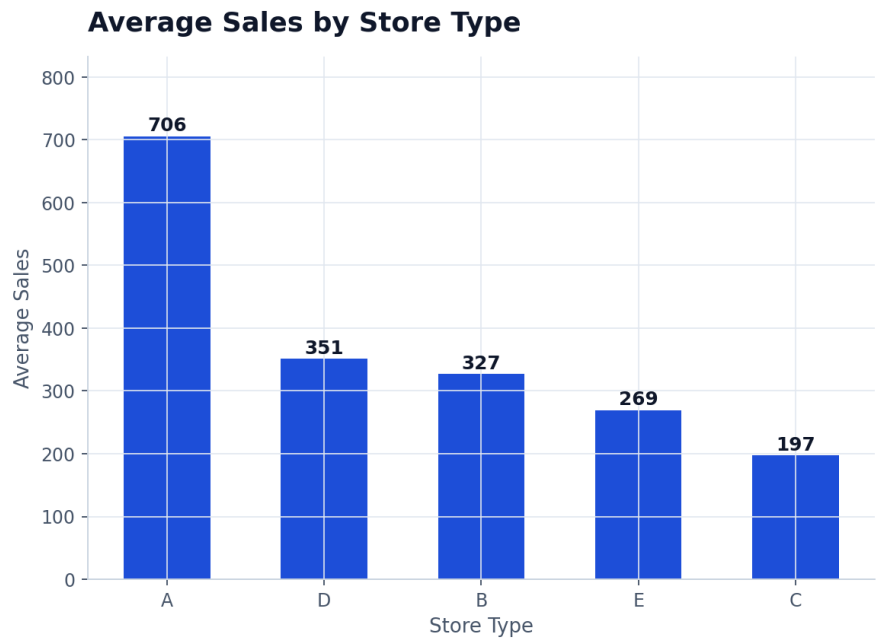


Figure 8. Average sales by store type (A–E). The relationship is not simply ordinal: Type D outperforms Type B and E.

Store type A stores are the strongest performers by a wide margin (705.9 average units), while type C is weakest (197.3). Notably the ordering is not a clean $A > B > C > D > E$ ramp — type D (351.0) outperforms both B (326.7) and E (269.1) — indicating that “type” functions more as a format/segment label than a simple size ranking, which is exactly why the model treats it as an unordered categorical feature (one-hot encoded) rather than an ordinal one.

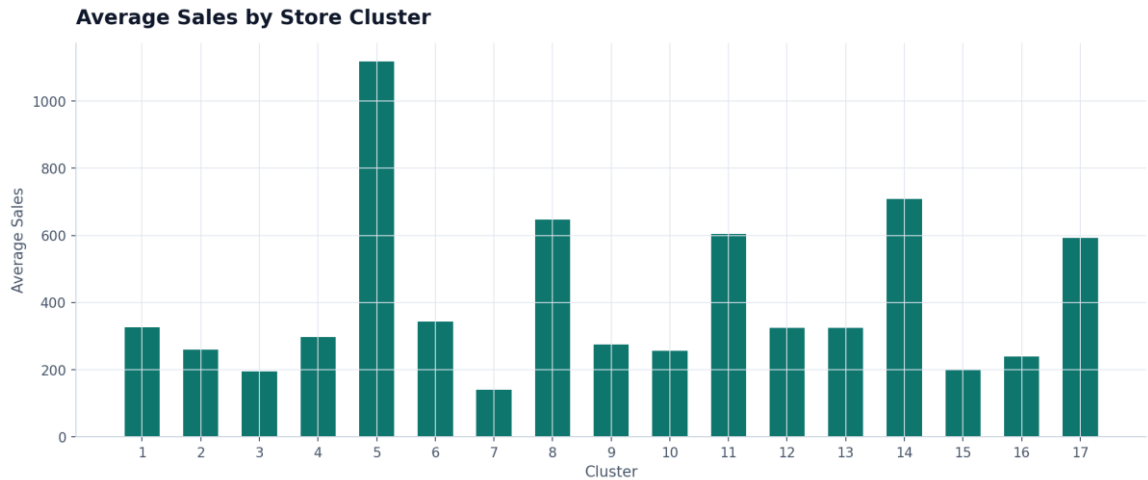


Figure 9. Average sales by store cluster (1–17). Cluster 5 is a clear outlier on the high side.

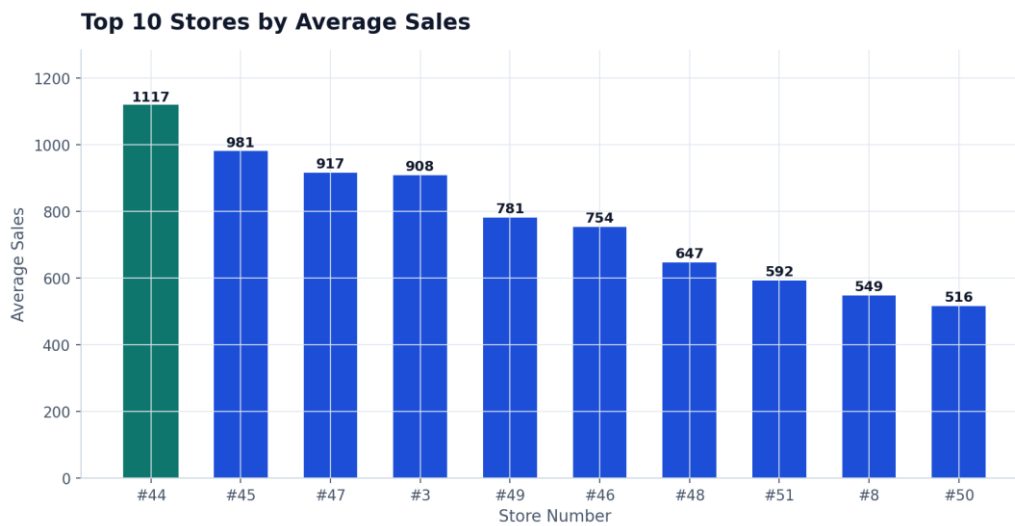


Figure 10. Top 10 stores by average sales. Store #44 leads the chain.

Store #44 is the single highest-performing store in the chain (1,117.3 average units), consistent with it driving Cluster 5's outsized cluster average. The next four stores (#45, #47, #3, #49) all fall between roughly 780 and 980 units — a distinct top tier well above the chain median.

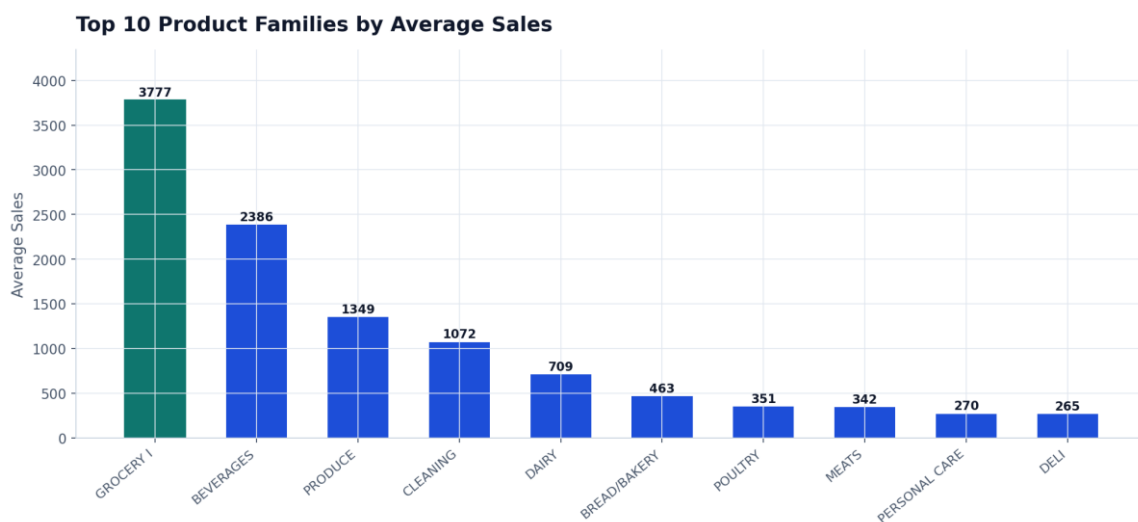


Figure 11. Top 10 product families by average sales. GROCERY I dominates.

GROCERY I is by far the largest product family by average sales (3,777.0 units), roughly 1.6× the second-place BEVERAGES (2,385.8) and 2.8× third-place PRODUCE (1,349.4). This concentration matters for business framing (Section 13): forecast accuracy on a handful of high-volume families disproportionately drives total inventory and revenue impact.

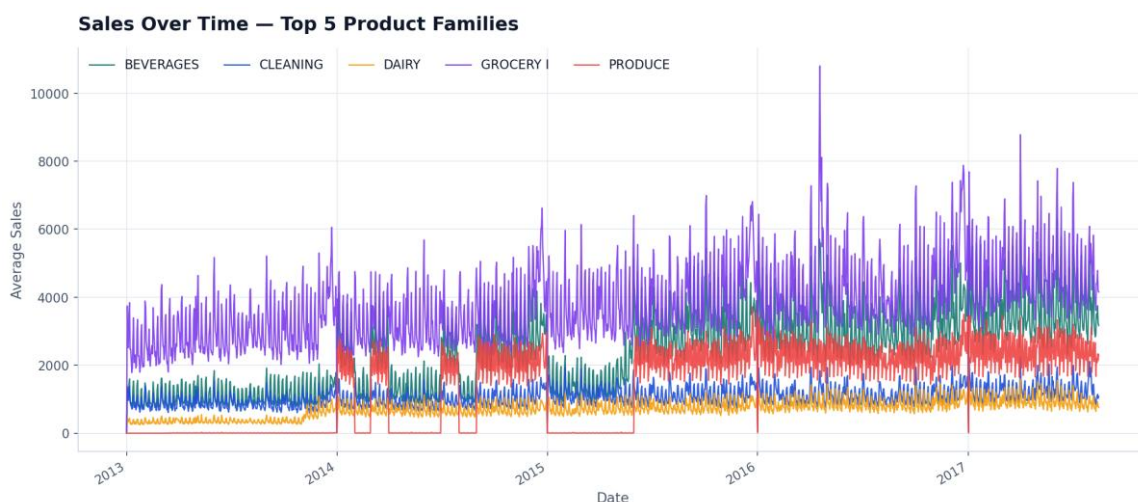


Figure 12. Daily average sales over time for the five top-selling product families, shown together to compare their trends and volatility.

City- and state-level aggregations were not separately charted in the notebook's exploratory pass — noted here plainly rather than glossed over — though both `city` and `state` are retained as one-hot-encoded categorical inputs to every row-level model, so the information is still available to the models even where it was not separately visualized during EDA.

4.8 Correlation Analysis

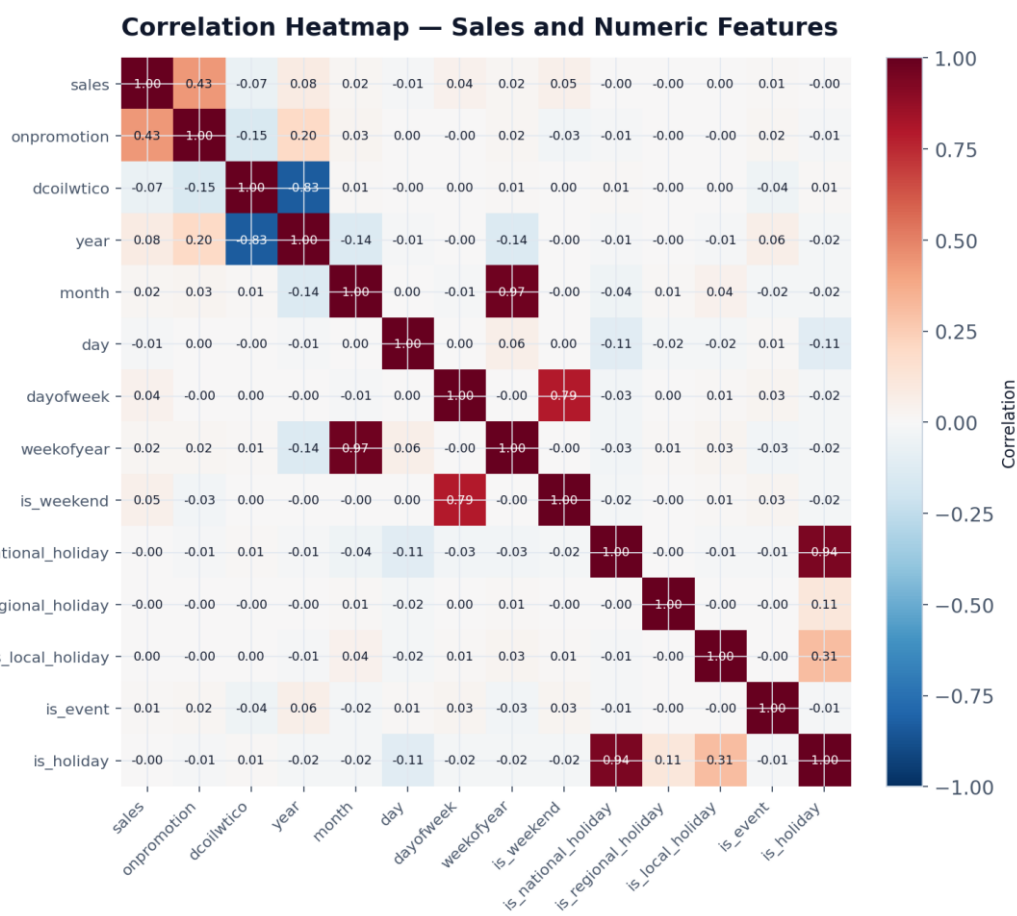


Figure 13. Correlation heatmap across sales and the numeric candidate features.

The heatmap corroborates the pairwise findings above in one view: **onpromotion** is the strongest linear correlate of **sales** among the numeric features, followed by weaker relationships for **transactions** and calendar fields, with **dcoilwtico** and the individual holiday-type flags showing only marginal linear association with sales in isolation. This is an expected and healthy pattern for tree-based modeling: XGBoost and Random Forest do not depend on strong pairwise linear correlation to exploit a feature, since they can capture non-linear interactions (for example, an oil-price level combined with a specific month or store cluster) that a correlation coefficient alone cannot reveal.

5. Advanced Time-Series Analysis

Beyond cross-sectional EDA, the notebook aggregates sales into a single national daily series (summing `sales` across all stores and families for each calendar date) to study trend, seasonality, and stationarity — the properties that determine whether, and how, a classical time-series model like SARIMA can be applied.

5.1 The Daily Aggregate Series

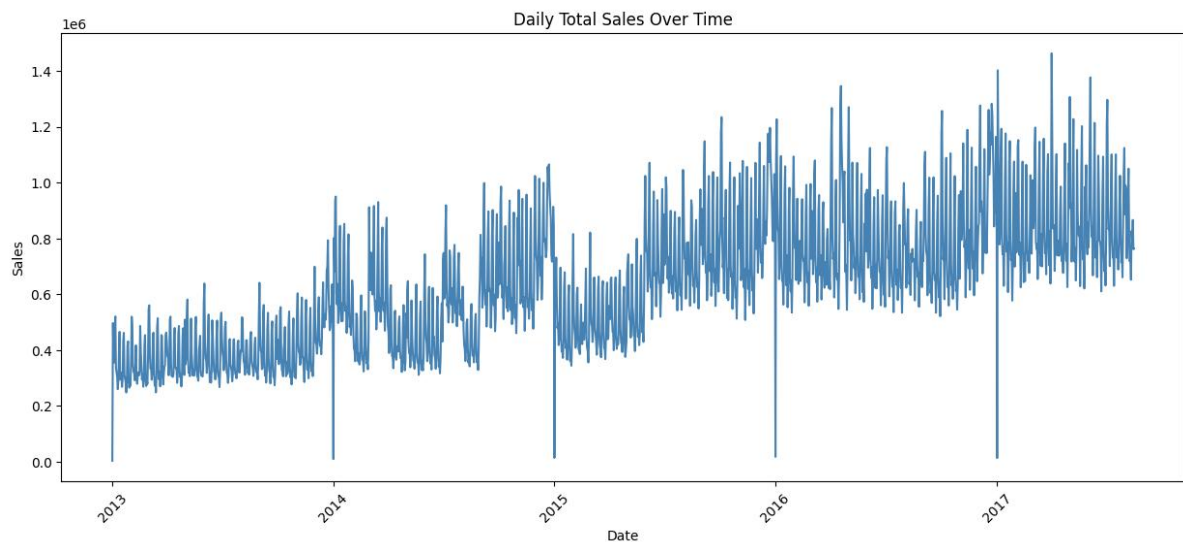


Figure 14. Daily total sales across all stores and families, January 2013 – August 2017.

The aggregated series opens on a striking data point: January 1, 2013 (New Year's Day) records only \$2,512 in national daily sales, against \$496,092 the very next day — an almost 200× jump that reflects widespread store closures or sharply reduced hours on the holiday itself, followed by an immediate return to a normal trading day. This single pair of values is a useful, concrete illustration of just how large calendar effects can be at the daily level, and why calendar/holiday features are essential inputs rather than an optional refinement.

5.2 Trend and Seasonal Decomposition



Figure 15. Additive seasonal decomposition (period = 365 days) of the daily aggregate sales series into observed, trend, seasonal, and residual components.

An additive `seasonal_decompose` (statsmodels, period = 365) separates the series into a long-run trend component that rises gently but steadily across the four-and-a-half-year window, a repeating annual seasonal component consistent with the month-level pattern already observed in Section 4.3, and a residual component that captures the sharper day-to-day and event-driven swings (promotions, individual holidays) that an annual seasonal term cannot absorb on its own.

5.3 Stationarity Testing (Augmented Dickey-Fuller)

[Code block: stationarity testing] Stationarity — a stable mean and variance over time — matters because most classical time-series models, including SARIMA, are built on the assumption (or are explicitly designed to remove the violation) that the series they model does not systematically drift or trend. Fitting such a model directly to a non-stationary series risks spurious parameter estimates and unstable forecasts. The Augmented Dickey-Fuller (ADF) test was used to check this formally, both on the raw aggregated daily series and after first-order differencing.

Series	ADF Statistic	p-value	Conclusion ($\alpha = 0.05$)
Original daily aggregate	-2.616	0.0897	Fail to reject $H_0 \rightarrow$ Non-stationary
After first-order differencing	-11.498	4.57×10^{-21}	Reject $H_0 \rightarrow$ Stationary

Table 3

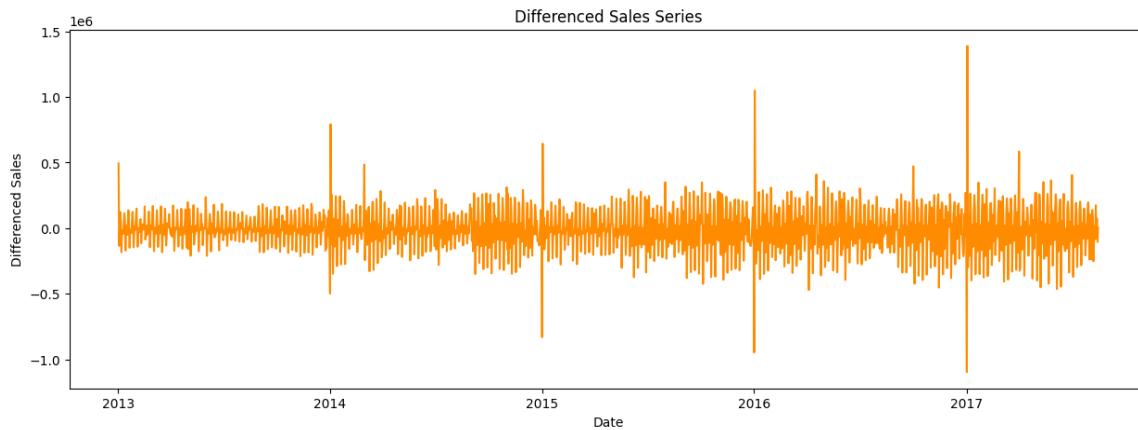


Figure 16. Daily aggregate sales after first-order differencing — the transformation that renders the series stationary.

The original series fails to reject the null hypothesis of a unit root at the conventional 5% threshold ($p = 0.090$), confirming the visibly upward-drifting trend already seen in the decomposition. A single first-order difference is enough to produce an overwhelmingly stationary series ($p \approx 4.6 \times 10^{-21}$). This single result directly justifies the $d=1$ term chosen for the SARIMA specification in Section 7, and — together with the clear 7-day cycle documented in Section 4.2 — the $D=1$, $s=7$ seasonal-differencing term.

5.4 Why a Time-Aware Approach Is Necessary

Two independent pieces of evidence in this project point to the same conclusion. First, the ADF results above confirm the series is non-stationary in levels, meaning a model (or a validation scheme) that implicitly assumes a stable statistical regime over the full 2013–2017 window will misrepresent it. Second, and more fundamentally for model validation, sales at any given date are influenced by very recent history (a promotion that started yesterday, a holiday last week) and by the broader macro/seasonal regime in effect at the time (the oil-price level, the calendar year) — information a model can only be fairly evaluated on if its test period is chronologically after, never intermixed with, its training period. This is exactly why the modeling in Section 8 uses a chronological train/test split and `TimeSeriesSplit` cross-validation instead of ordinary random splitting.

6. Feature Engineering

[Code block: feature engineering] The notebook engineers a substantially larger feature set than the one ultimately used for modeling. Documenting both — what was built and what was kept — is important for an honest account of the project's methodology.

6.1 Time-Based Features

Calendar fields were derived directly from the `date` column: `year`, `month`, `day`, `dayofweek`, `weekofyear`, `quarter`, `dayofyear`, and boolean flags for month/quarter/year start and end. A derived `is_weekend` flag (`dayofweek ≥ 5`) captures the weekly seasonality documented in Section 4.2.

6.2 Cyclical Encodings

Sine/cosine transforms of month and day-of-week (`month_sin/month_cos`, `dow_sin/dow_cos`) were engineered to represent calendar cyclicity without an artificial “January is far from December” discontinuity that a raw integer encoding would imply to a linear model. These were built and available but — as noted below — not included in the final feature list actually passed to the models, since the tree-based models used in this project (Random Forest, XGBoost) can already partition on the raw month/day-of-week integers without needing an explicit cyclical transform.

6.3 Lag and Rolling-Window Features

Per (store_nbr, family) group, the notebook computes lagged sales at 1, 7, 14, and 28 days (`lag_1...lag_28`) and rolling 7-day and 30-day means of sales (each shifted by one day before the rolling window is applied, to avoid leaking the current day's own value into its own feature). Equivalent lag/rolling features were also built for the `transactions` series (`trans_lag1`, `trans_roll7`, `trans_change`) and for oil prices (`oil_change`, `oil_pct_change`, `oil_roll7`, `oil_roll30`).

6.4 Holiday-Related Features

`is_national_holiday`, `is_regional_holiday`, `is_local_holiday`, `is_event`, and the combined `is_holiday` flag were derived from the four holiday lookup tables described in Section 3.5. These five flags are the only holiday-related features that made it into the final production feature set.

6.5 Promotion-Related and External-Signal Features

`promo_binary` (onpromotion > 0), `promo_log` (log1p of the promotion count, to compress its heavy tail), and `promo_weekend` (an explicit interaction between promotion and the weekend flag) were engineered, alongside an `oil_transactions` interaction term (oil price × transaction count).

6.6 What Was Ultimately Used — and What Was Not

The final modeling feature set — confirmed both in the notebook's model-training cells and, independently, in the shared `feature_engineering.py` module that powers the deployed API and dashboard — comprises 20 features: 16 numeric (store_nbr, onpromotion, dcoiltwtico, transactions, year, month, day, dayofweek, weekofyear, is_weekend, is_holiday, is_national_holiday, is_regional_holiday, is_local_holiday, is_event, cluster) and 4 one-hot-encoded categorical fields (family, city, state, type).

◆ Engineered but not used — and a plausible reason why

A meaningful share of the engineered features described above (the cyclical month/day-of-week encodings; the lag_1/lag_7/lag_14/lag_28 and rolling7/rolling30 sales features; promo_binary, promo_log, and promo_weekend; the oil and transaction derivative features oil_change/oil_pct_change/oil_roll7/oil_roll30/oil_transactions and trans_lag1/trans_roll7/trans_change; and the manually label-encoded family_code/city_code/state_code/type_code/cluster_code) were computed during exploration but do not appear in the final 20-feature list. This is stated plainly here as a factual gap rather than glossed over. The most defensible reason — offered as a reasoned inference rather than a claim found verbatim in the notebook — is operational: lag and rolling-sales features require access to very recent ground-truth sales at prediction time, which is straightforward inside a historical notebook but adds real complexity to a live API that must serve a prediction for an arbitrary future store/family/date without necessarily having same-day or prior-day sales already loaded. Building the deployed feature pipeline (see [feature_engineering.py](#)) without a lag dependency keeps the `/predict` endpoint self-contained and simple to operate, at the cost of leaving on the table some of the predictive power that recent-sales momentum features typically provide in demand forecasting. Similarly, one-hot encoding the raw categorical columns inside the scikit-learn pipeline is a cleaner and more standard choice than reusing the manually assigned integer category codes, which would risk implying a false ordinal relationship between, say, city codes, particularly for the Linear Regression baseline. Both omissions are revisited as concrete, low-effort future improvements in Section 15.

Feature Group	Included in Final Model	Engineered but Not Used
Calendar / time	year, month, day, dayofweek, weekofyear, is_weekend	quarter, dayofyear, month/quarter/year start-end flags, cyclical sin/cos encodings
Promotions	onpromotion (raw count)	promo_binary, promo_log, promo_weekend
Sales history	— none	lag_1, lag_7, lag_14, lag_28, rolling7, rolling30
Oil / macro	dcoilwtico (level)	oil_change, oil_pct_change, oil_roll7, oil_roll30, oil_transactions
Transactions	transactions (raw count)	trans_lag1, trans_roll7, trans_change
Holidays	is_holiday, is_national_holiday, is_regional_holiday, is_local_holiday, is_event	— none (full holiday set was carried through)
Categoricals	family, city, state, type (one-hot)	family_code, city_code, state_code, type_code, cluster_code (label-encoded versions)
Store attributes	store_nbr, cluster	— none

Table 4

7. Model Development

[Code block: model training] Four models spanning three methodological families — a linear baseline, two tree-ensemble methods, and a classical statistical time-series model — were trained and evaluated under a common, chronologically correct validation philosophy.

7.1 Baseline: Linear Regression

A plain `LinearRegression` (scikit-learn) was fit on the full 20-feature set (categoricals one-hot encoded via the shared `ColumnTransformer`) to establish a baseline. Linear Regression was chosen specifically as a baseline rather than a candidate for production: it assumes additive, linear relationships between features and the target, which is a poor structural fit for a target as skewed and interaction-heavy as retail sales (for instance, the promotion effect in Section 4.5 clearly interacts with store and family, not just adding a constant uplift).

7.2 Tuned Random Forest Regressor

A `RandomForestRegressor` was tuned via `RandomizedSearchCV` (3 candidate combinations, cross-validated with a 2-fold `TimeSeriesSplit`) over a deliberately narrow grid — `n_estimators` $\in \{10, 20\}$, `max_depth` $\in \{6, 10\}$, with `min_samples_split=10` and `min_samples_leaf=5` fixed. Random Forest was included as a strong, robust non-linear benchmark that requires little tuning to perform reasonably well and is naturally resistant to overfitting through bagging.

◆ Honest note on search breadth

The tuning grid used for Random Forest is intentionally small (only 2 values each for tree count and depth, and only 3 of the possible 4 combinations were actually sampled). This is a reasonable, pragmatic choice given the ≈ 2.4 -million-row training set and the cost of fitting many deep forests, but it does mean the reported Random Forest result should be read as a solid, credible benchmark rather than that model's absolute ceiling — a wider search (more trees, greater depth, tuned `max_features`) would likely close some of the gap to XGBoost, at a meaningfully higher training-time cost. This trade-off is revisited in Section 15.

7.3 XGBoost Regressor (Final Model)

An `XGBRegressor` was configured with `n_estimators=300`, `learning_rate=0.05`, `max_depth=8`, `subsample=0.8`, `colsample_bytree=0.8`, and `objective='reg:squarederror'`. These hyperparameters were used directly (not grid/random-searched in the visible notebook flow) and are the same configuration subsequently hard-coded into the production retraining script (`retrain.py`), which is a good consistency signal between the exploratory notebook and the operational codebase. XGBoost is well suited to this problem for three structural reasons: it handles non-linear feature interactions natively (relevant given the promotion $\times$ store $\times$ family patterns above), it is robust to the heavy right skew in the target without requiring an explicit log transform, and gradient boosting's sequential error-correction typically outperforms a single bagged forest on tabular data of this size and feature complexity — which is exactly what the results in Section 10 confirm.

7.4 SARIMA (Classical Time-Series Benchmark)

A $\text{SARIMAX}(1, 1, 1) \times (1, 1, 1, 7)$ model was fit to the aggregated national daily sales series (summed across all stores and families) rather than to the row-level store-family-day data the other three models use. This is an important methodological distinction, not an oversight: SARIMA is a univariate model of a single time series, so it is structurally suited to forecasting one aggregate series (e.g., total daily company-wide sales) rather than millions of parallel, cross-sectionally distinct store-family series. It was included specifically to test whether a classical, purely time-series approach — with no store, family, promotion, or holiday covariates at all — could compete with the covariate-rich machine learning models, and, as Section 10 shows, it could not: without access to the store/family/promotion signals that drive most of the variance in this dataset, SARIMA's aggregate-only view is fundamentally under-informed relative to the other three models.

7.5 Why Each Model Fits — or Does Not Fit — the Problem

Model	Structural Fit for This Problem
Linear Regression	Fast, fully interpretable baseline; poor fit for a skewed target with strong non-linear interactions (promotion $\times$ store $\times$ family).
Random Forest	Captures non-linearity and interactions via bagged trees; robust to overfitting; less sample-efficient than boosting on this size of tabular data.
XGBoost	Sequential boosting captures non-linear interactions and residual structure other models miss; handles skew and mixed feature types well; matches this project's best result.
SARIMA	Well suited to a single aggregate series with clear weekly/annual seasonality (confirmed in Section 5); structurally cannot use store-, family-, or promotion-level covariates, capping its achievable accuracy on this task.

Table 5

8. Training and Validation Strategy

8.1 Chronological Splitting

[Code block: validation strategy] All three row-level models (Linear Regression, Random Forest, XGBoost) share an identical chronological 80/20 split of the 3,000,888-row modeling table: the first 2,400,710 rows (2013-01-01 to 2016-09-12) form the training set, and the remaining 600,178 rows (2016-09-12 to 2017-08-15, roughly the final eleven months of data) form the held-out test set. SARIMA uses the same cut date applied to the aggregated daily series (1,350 training days vs. 338 test days).

8.2 Why Random Splitting Would Be Incorrect

A random row-wise train/test split would place some days from, say, August 2017 into the training set while holding out days from 2014 for testing — which would let the model implicitly “see the future” relative to whatever it is being asked to predict, and would badly overstate real-world accuracy, since lag/rolling and lookup-style features (and, more subtly, the very fact that the model has already learned the shape of a period statistically adjacent to the test point) leak information across a random split in a time series that a genuinely future deployment would never have. A chronological split is the only design that honestly answers the operationally relevant question: “how accurately would this model have forecast sales it had never seen, using only data available before that point?”

8.3 Leakage Prevention

Beyond the chronological split itself, two additional choices protect against leakage: rolling and lag features (Section 6.3), where used, were always computed with an explicit `.shift(1)` before the rolling window so that a given day's own sales value can never contribute to its own feature; and the held-out test set was never touched during feature selection, hyperparameter tuning, or cross-validation — all tuning and cross-validation happens strictly inside the training partition.

8.4 TimeSeriesSplit and Rolling-Origin Validation

Within the training partition, `sklearn.model_selection.TimeSeriesSplit` (3 folds for Linear Regression and XGBoost, 2 folds for the Random Forest random search) was used for cross-validation. Unlike ordinary k-fold CV, `TimeSeriesSplit` only ever validates on data chronologically after the fold's training data, preserving the same no-look-ahead discipline within cross-validation that the outer train/test split enforces overall.

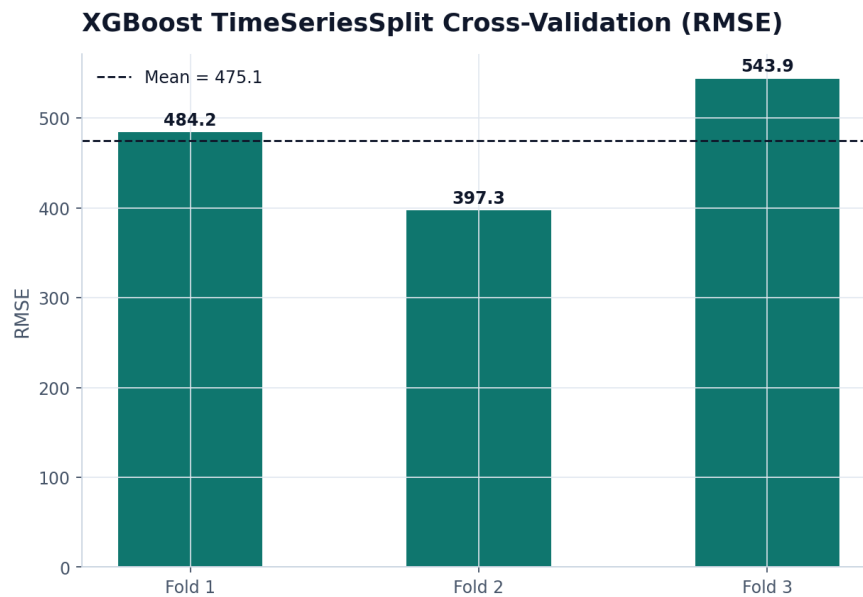


Figure 17. XGBoost TimeSeriesSplit cross-validation RMSE across three expanding-window folds.

XGBoost's three CV folds returned RMSE scores of 484.2, 397.3, and 543.9 (mean 475.1, std 60.2) — a reasonably tight spread given the fold-to-fold seasonal and volume differences inherent to an expanding time window, and notably higher than the final held-out RMSE of 331.6, which is expected since the final model is trained on the full training partition (more data) and evaluated once on a single, specific test window rather than averaged across three smaller, earlier ones.

SARIMA received an additional, separate rolling-origin (walk-forward) cross-validation, with three sequential 30-day validation folds, each using an expanding training window ending immediately before that fold's validation period:

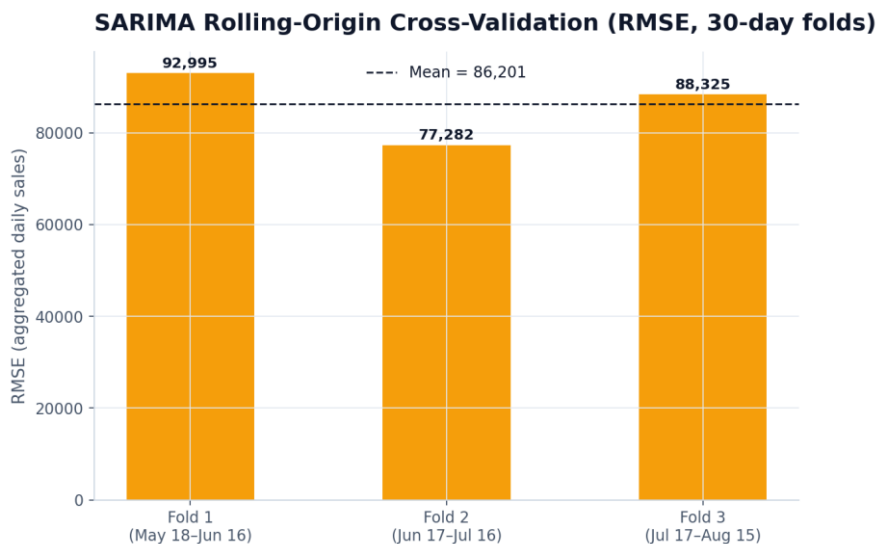


Figure 18. SARIMA rolling-origin cross-validation RMSE across three sequential 30-day folds (aggregated daily sales).

Fold	Train End	Validation Window	MAE	RMSE
1	2017-05-17	2017-05-18 – 2017-06-16	70,299.2	92,994.8
2	2017-06-16	2017-06-17 – 2017-07-16	64,278.5	77,281.7

Fold	Train End	Validation Window	MAE	RMSE
3	2017-07-16	2017-07-17 – 2017-08-15	58,584.0	88,325.1

Table 6

This gives SARIMA arguably the most rigorous validation of any single model in the project — three independent, sequential out-of-sample windows rather than one — and its mean rolling RMSE (86,200.5, std 8,069.1) is consistent with its single final-holdout RMSE (131,228.4 on a different, slightly larger test span), giving reasonable confidence that its evaluated performance is stable rather than a one-off fluke of a particular test window, even though its absolute accuracy trails the row-level models substantially (Section 10).

8.5 Assessing Generalization

Generalization was assessed through the consistent gap-check between cross-validation RMSE and final test RMSE described above, and by inspecting actual-vs-predicted and residual plots for every model (Sections 10 and 11) rather than trusting a single scalar metric in isolation — a residual pattern that only shows up visually (for example, systematic under-prediction at high sales volumes, discussed in Section 11) would not necessarily be visible from RMSE or R^2 alone.

9. Evaluation Metrics

Every model in this project is scored on the same four metrics, each surfacing a different facet of forecast quality.

9.1 Mean Absolute Error (MAE)

MAE is the average absolute difference between predicted and actual sales, in the original unit-sales scale. It is the most directly business-interpretable metric in the set: an MAE of 96.4 units (the final XGBoost result) means that, on a typical store-family-day, the forecast misses actual sales by about 96 units in either direction — a figure a merchandising or inventory planner can reason with directly.

9.2 Mean Squared Error (MSE)

MSE squares each error before averaging, which disproportionately penalizes large misses. It is less directly interpretable in business terms (its units are squared sales), but it is the objective XGBoost's `reg:squarederror` loss directly optimizes, and it is the basis for RMSE below.

9.3 Root Mean Squared Error (RMSE)

RMSE is the square root of MSE, returning the error to the original sales unit scale while still weighting large errors more heavily than MAE does. Because RMSE is sensitive to a small number of very large misses, and because this dataset's target is heavily right-skewed (Section 4.1), RMSE and MAE are read together in this project: a model with a low MAE but a comparatively high RMSE would indicate that it is generally accurate but occasionally very wrong on the highest-volume rows — precisely the rows that matter most for inventory decisions.

9.4 R^2 (Coefficient of Determination)

R^2 reports the share of variance in actual sales explained by the model, on a scale where 1.0 is a perfect fit and 0.0 matches a naive mean-only baseline. It is the most convenient single number for cross-model ranking because, unlike MAE/RMSE, it does not depend on the scale of the target — which matters in this project specifically because SARIMA's target (aggregated daily company-wide sales) lives on a completely different numeric scale from the other three models' target (individual store-family-day sales); R^2 is thus the fairest common ground for the four-way comparison in Section 10.

9.5 Business Reading of the Metrics

- MAE answers: “on a normal day, how far off is the forecast, in units I can reason about for reordering?”
- RMSE answers: “how much should I worry about the occasional big miss, particularly on high-volume products?”
- R^2 answers: “overall, how much of the real variation in demand is this model actually capturing, versus just following a flat average?”
- Together, they support a nuanced deployment decision: a model chosen purely to minimize MAE might not be the safest choice if its RMSE (or its residual pattern) reveals it occasionally makes very large errors on the most inventory-critical, high-volume rows.

10. Model Comparison

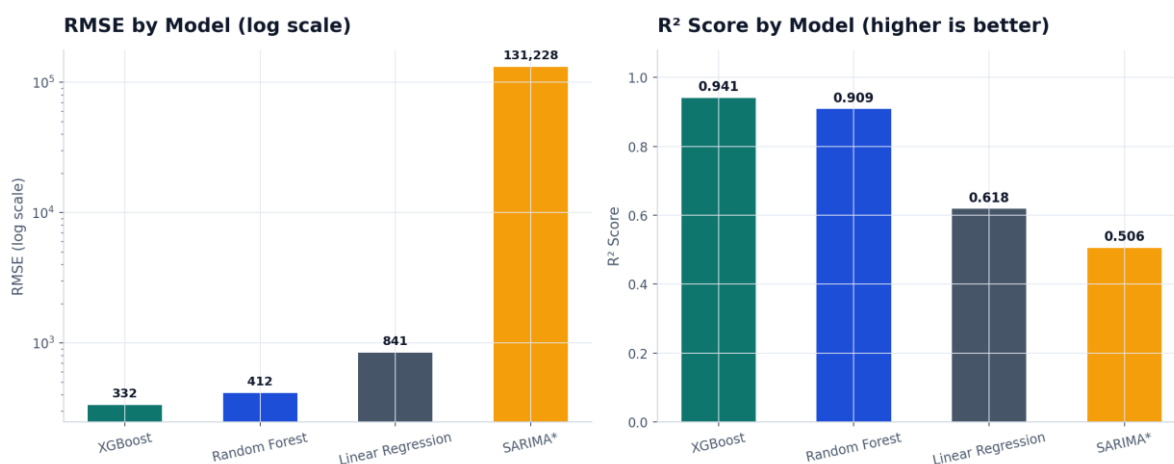


Figure 19. RMSE (log scale, left) and R^2 (right) across all four models. XGBoost leads on both metrics; SARIMA's RMSE is plotted on a log axis because it is measured on aggregated, not row-level, sales.

Model	Validation Approach	MAE	MSE	RMSE
XGBoost Regressor	80/20 chronological + 3-fold TimeSeriesSplit	96.36	109,965.41	331.61
Tuned Random Forest	80/20 chronological + 2-fold TimeSeriesSplit RandomizedSearchCV	144.15	169,335.30	411.50
Linear Regression	80/20 chronological + 3-fold TimeSeriesSplit	318.22	707,213.52	840.96
SARIMA(1,1,1)(1,1,1,7)*	80/20 chronological (aggregated daily) + 3-fold rolling-origin CV	86,067.66	1.72×10^{10}	131,228.40

Table 7

* SARIMA's MAE/MSE/RMSE are measured on aggregated company-wide daily sales (a series in the hundreds-of-thousands of units), not on individual store-family-day rows like the other three models, so its absolute error figures are on a different numerical scale and are not directly comparable to the other rows. R^2 is scale-independent and is the fair basis for cross-model comparison.

10.1 Strengths and Weaknesses

Model	Strengths	Weaknesses	Final Rank
XGBoost	Best accuracy on every metric; handles non-linearity, interactions, and skew natively; hyperparameters already mirrored in the production retraining script	Less interpretable than linear regression; fixed hyperparameters were not exhaustively searched	1st
Random Forest	Strong, robust non-linear performance; resistant to overfitting via bagging; simple to reason about	Clearly behind XGBoost on every metric; tuning grid was narrow, so its ceiling is likely somewhat understated	2nd
Linear Regression	Fast, fully transparent, easy to explain to non-technical stakeholders	Structurally too simple for a skewed, interaction-heavy target; by far the weakest of the three row-level models	3rd
SARIMA	Rigorous, dedicated rolling-origin validation; genuinely useful as a pure time-series sanity	Cannot use store-, family-, or promotion-level information at all; weakest R^2 of the four; not	4th

Model	Strengths	Weaknesses	Final Rank
	check; no dependency on store/product covariates	comparable in absolute error terms to the row-level models	

Table 8

10.2 Actual vs. Predicted and Residual Diagnostics — All Four Models

Beyond the summary metrics, the notebook plots daily aggregated actual-vs-predicted sales and a residual scatter (predicted value vs. residual) for every model over its test period. These are reproduced together here for direct visual comparison.

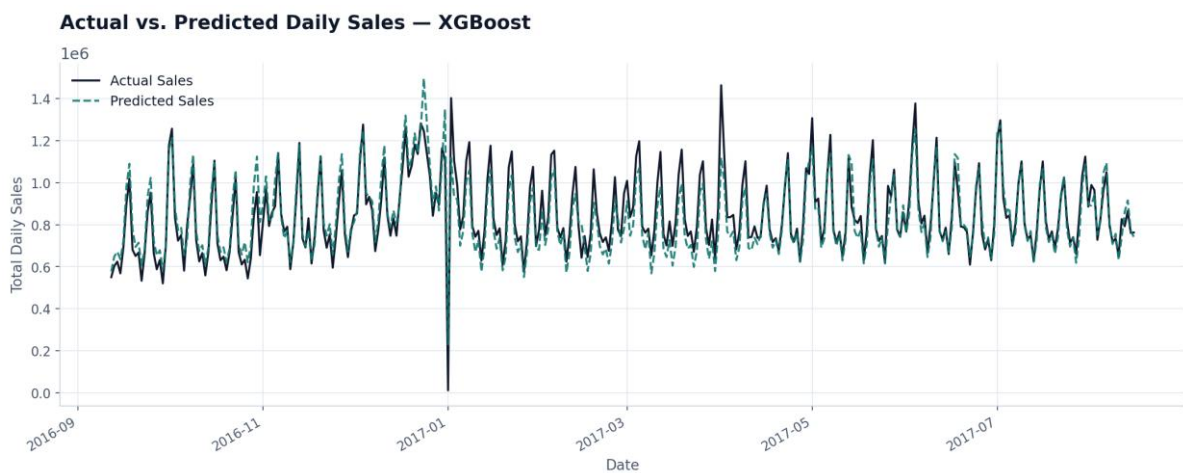


Figure 20. Actual vs. predicted daily sales — XGBoost. Predicted values track the actual series closely, including its sharper peaks.

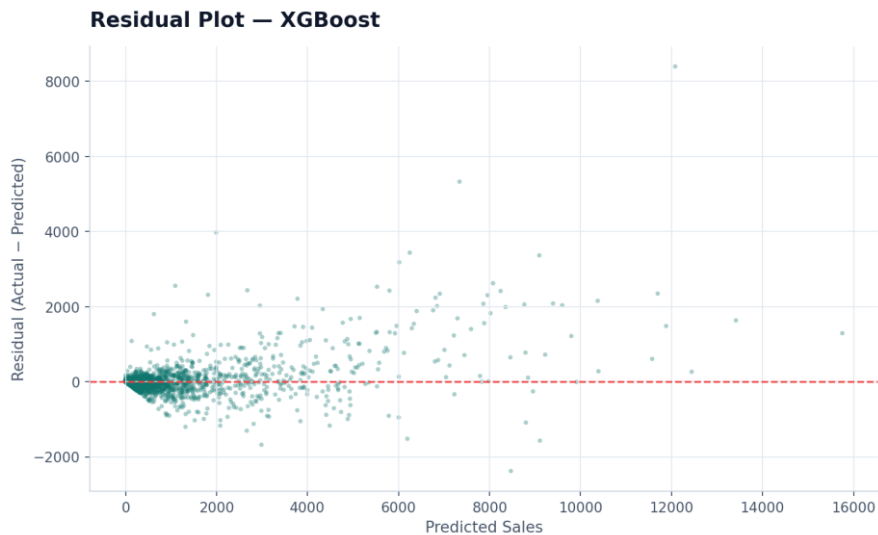


Figure 21. Residual plot — XGBoost. Residuals cluster tightly around zero for typical volumes, widening (a classic funnel shape) at the highest predicted values.

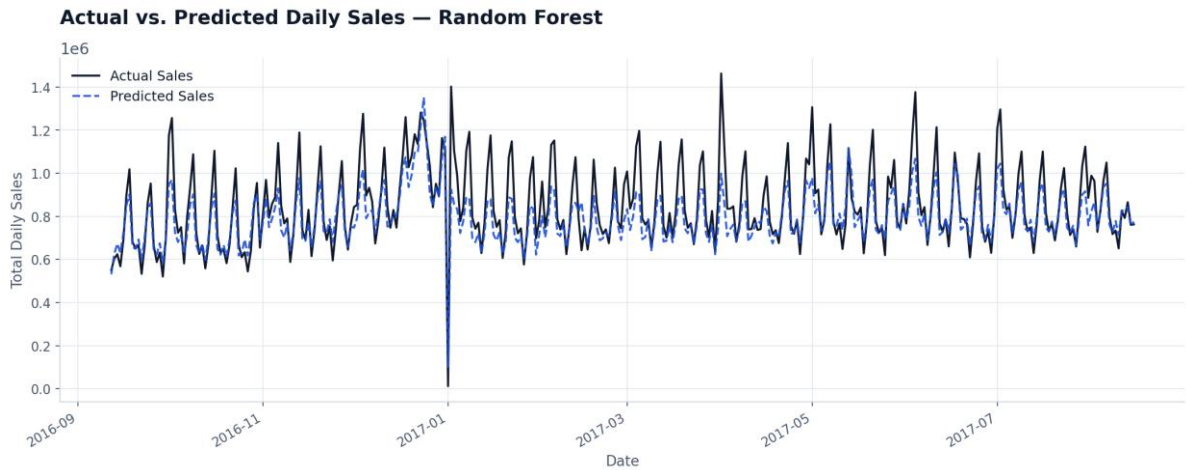


Figure 22. Actual vs. predicted daily sales — Random Forest. Tracks the broad shape of demand but visibly under-shoots the sharpest peaks.

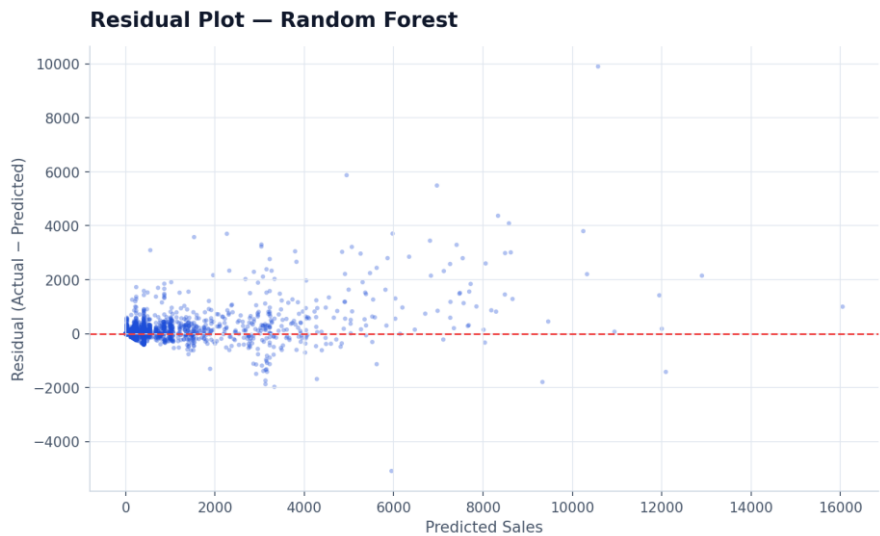


Figure 23. Residual plot — Random Forest. A similar but wider funnel pattern than XGBoost, consistent with its higher RMSE.

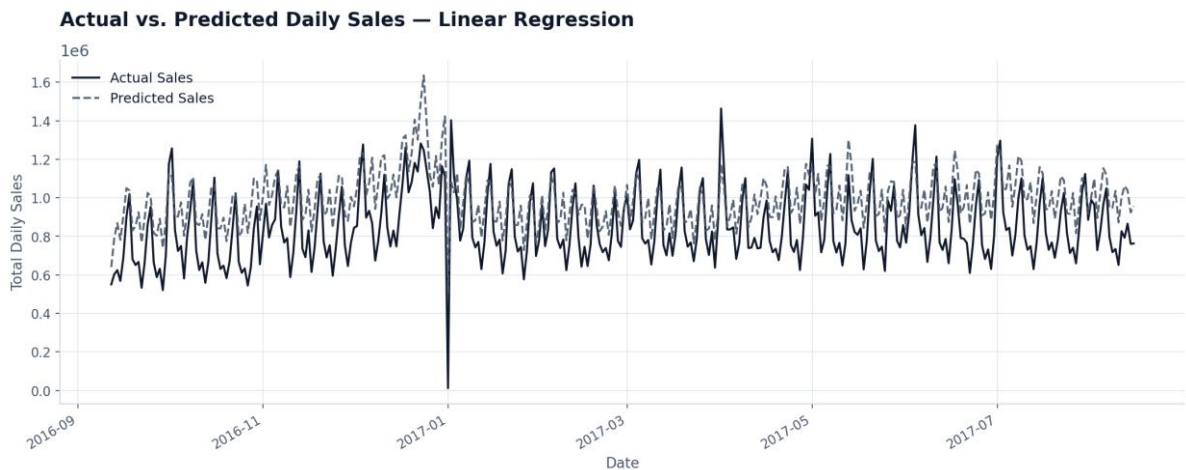


Figure 24. Actual vs. predicted daily sales — Linear Regression. Visibly smooths over peaks and troughs the other models capture.

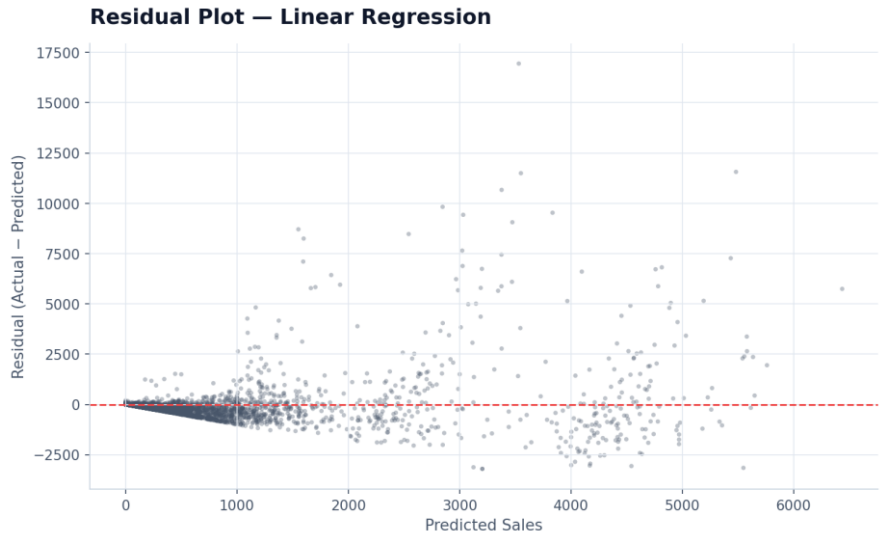


Figure 25. Residual plot — Linear Regression. Much wider and more structured residual spread, consistent with a poor fit for a non-linear, skewed target.

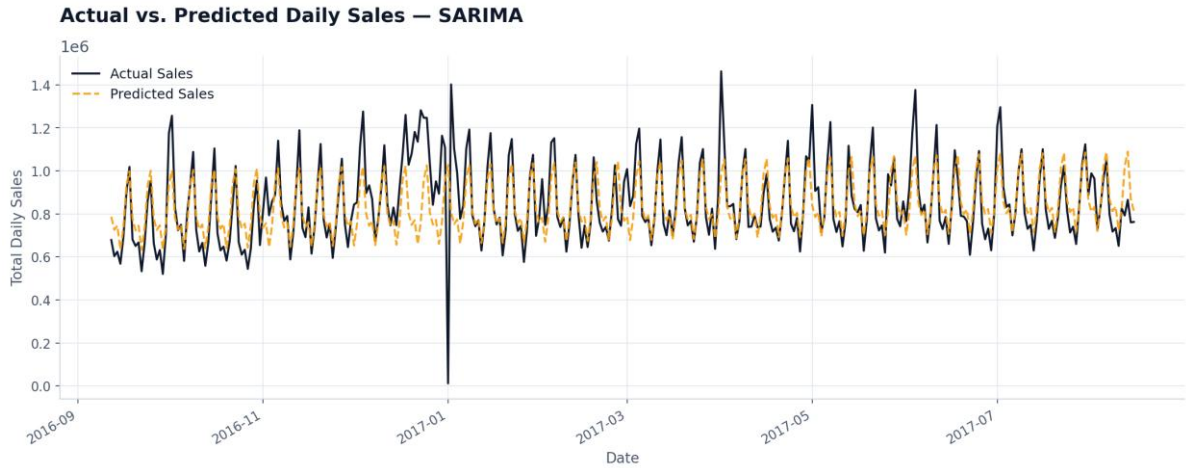


Figure 26. Actual vs. predicted daily sales — SARIMA (aggregated series). Captures the general level and weekly rhythm but misses much of the sharper day-to-day and promotional variation the covariate-aware models pick up.

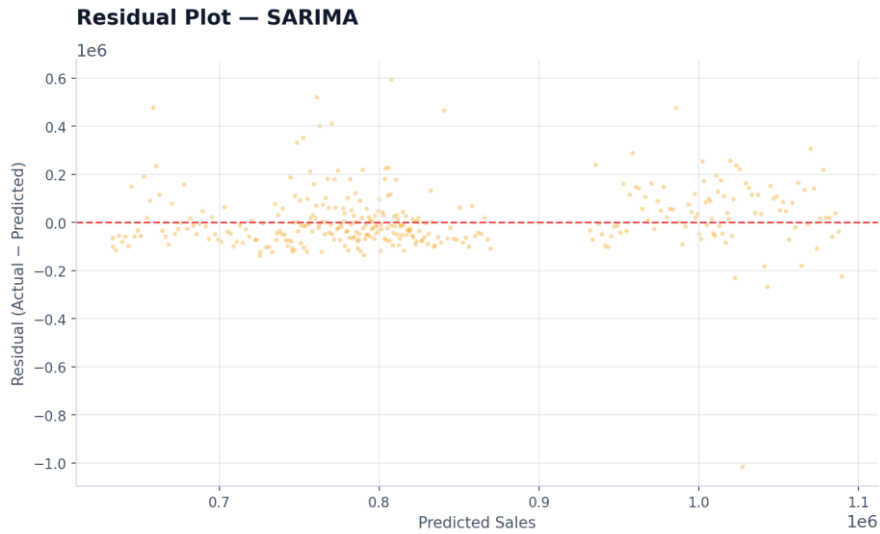


Figure 27. Residual plot — SARIMA. Residuals are large in absolute terms (consistent with the aggregated scale) and show more visible autocorrelated structure than the row-level models' residuals.

The residual funnel visible in all three row-level models — tight near zero for low-to-moderate predicted sales, widening for the highest predicted values — is an honest, common pattern for a right-skewed retail target: absolute errors naturally grow alongside the size of the number being predicted. XGBoost's funnel is visibly the narrowest of the three, consistent with its lead on every summary metric.

11. Final Model Selection

11.1 Why XGBoost Was Chosen

XGBoost was selected as the final production model because it recorded the lowest MAE (96.36), the lowest RMSE (331.61), and the highest R^2 (0.9407) of all four candidates — a clean sweep across every metric tracked, not a mixed result requiring a judgment call between competing metrics.

11.2 Why It Outperformed the Others

XGBoost's advantage over Random Forest (24% lower RMSE) is consistent with gradient boosting's core mechanism: each new tree is fit specifically to correct the residual errors of the ensemble built so far, which tends to extract more signal from a large, feature-rich tabular dataset than a bagged forest of independently grown trees, especially once interaction effects (promotion $\times$ store $\times$ family, oil price $\times$ year, and similar) are involved. Its advantage over Linear Regression (61% lower RMSE) reflects the latter's structural inability to model the non-linear, interaction-heavy relationships documented throughout the EDA. Its advantage over SARIMA is of a different nature entirely — SARIMA is simply not given access to the store-, family-, or promotion-level covariates that drive most of the row-level variance in this dataset, so the comparison mainly demonstrates the value of a covariate-rich approach over a pure univariate time-series model for this particular forecasting problem.

11.3 Trade-offs Considered

- Accuracy vs. interpretability: XGBoost is materially less transparent than Linear Regression. This is judged an acceptable trade-off given the size of the accuracy gap and the fact that feature-importance and SHAP-style explanations (noted as a future improvement in Section 15) can partially restore interpretability without sacrificing accuracy.
- Accuracy vs. training cost: XGBoost with 300 estimators and max depth 8 is more expensive to train than Linear Regression and comparable to, or cheaper than, the Random Forest search given the latter's cross-validated hyperparameter search; in absolute terms, none of the three row-level models pose a practical training-time obstacle for periodic retraining (Section 12).
- Accuracy vs. operational simplicity: choosing not to depend on lag/rolling sales features (Section 6.6) sacrifices some achievable accuracy in exchange for a simpler, self-contained real-time API — a trade-off made once, for all row-level models equally, rather than one specific to XGBoost.

11.4 Suitability for Real-World Use

At $R^2 = 0.94$ and $MAE \approx 96$ units against an average of 358 units per row, the final model already delivers materially actionable accuracy for replenishment and staffing decisions, it is already wrapped in a versioned artifact (`xgboost_sales_forecasting_model.pkl`) served through a documented REST API, and it is already connected to a monitoring and retraining loop (Section 12) that can catch performance degradation before it becomes a business problem — the combination of accuracy and operational readiness is precisely why XGBoost, rather than a marginally more accurate but harder-to-operate alternative, is the right production choice here.

12. MLOps, Experiment Tracking, and Monitoring

[Code block: monitoring logic] This project treats deployment, monitoring, and retraining as first-class deliverables rather than an afterthought, implementing each as standalone, reusable modules (`app.py`, `monitoring.py`, `retrain.py`) rather than one-off notebook cells.

12.1 Experiment Tracking with MLflow

The final XGBoost run is logged to an MLflow experiment named “Sales Forecasting” under the run name `Final_XGBoost`: the model type is logged as a parameter, MAE/MSE/RMSE/R² are logged as metrics, and the fitted booster itself is logged as a versioned model artifact via `mlflow.xgboost.log_model`. The monitoring module additionally logs each periodic evaluation's metrics to a second MLflow experiment (“Sales_Forecasting_Monitoring”), and the retraining module logs every retraining attempt — deployed or not — to a third (“Sales_Forecasting_Retraining”), giving three cleanly separated, queryable histories: initial development, ongoing monitoring, and retraining decisions.

12.2 Drift Detection

`monitoring.py` implements `detect_data_drift`, which runs a two-sample Kolmogorov-Smirnov (KS) test, feature by feature, comparing the distribution of each numeric feature in the training partition against the held-out test partition (used here as a stand-in for “live” inference data). Applied to this project's own train/test split, 13 of the 16 numeric features register a statistically significant distributional shift at $p < 0.05$.

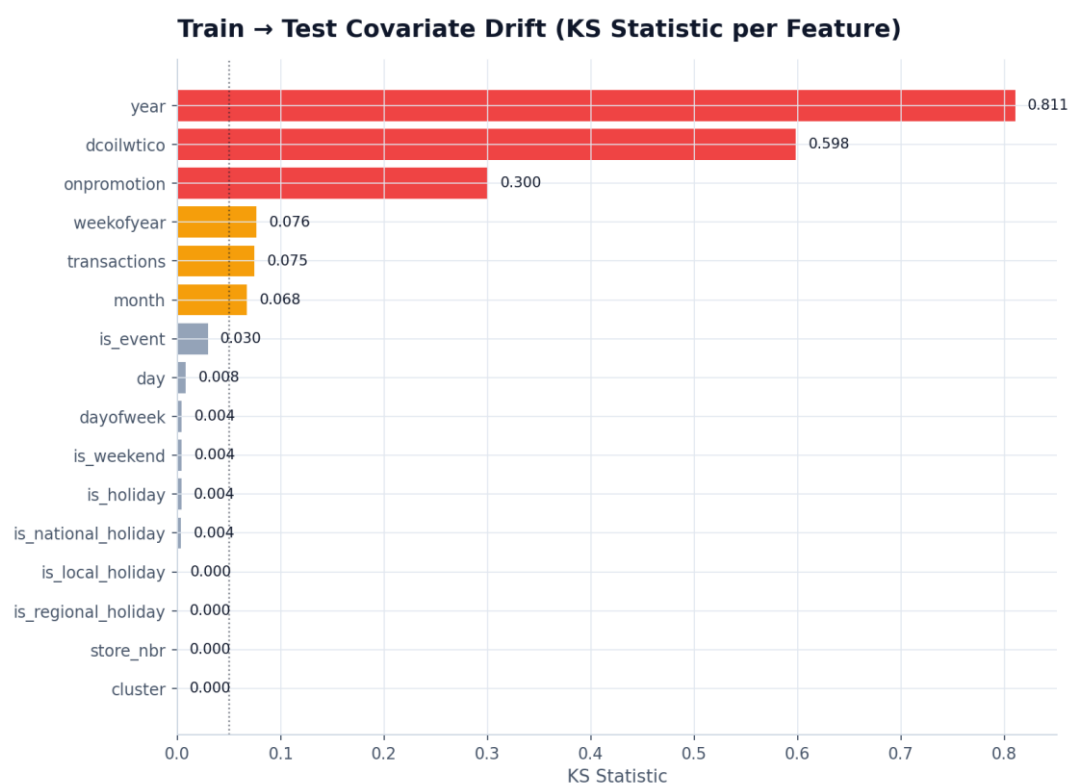


Figure 28. Kolmogorov-Smirnov drift statistic per feature, train vs. test partition. The dotted line marks the conventional $p < 0.05$ significance threshold; bars are colored by practical materiality, not just statistical significance.

◆ Statistically significant is not the same as practically meaningful

With roughly 2.4 million training rows and 600,000 test rows, the KS test has enormous statistical power — it will flag even a trivially small distributional difference as “significant” simply because the sample size makes the test extremely sensitive. Reading the KS statistics themselves (not just the significance flags) separates the genuinely important shifts from statistical noise. Three features show large, practically important KS statistics: `dcoilwtico` (KS = 0.598 — the mean oil price fell from \$72.63 in training to \$49.11 in test, directly reflecting the real 2014–2016 oil-price collapse discussed in Section 4.6); `onpromotion` (KS = 0.300 — mean promotion intensity rose roughly five-fold, from 1.47 to 7.14 items on promotion per row, between the training and test periods); and `year` (KS = 0.811 — an expected, mechanical consequence of a chronological split, not a meaningful signal in its own right). By contrast, features such as `dayofweek` (KS = 0.0045), `is_weekend` (0.0045), and `day` (0.0082) are technically “drifted” at $p < 0.05$ but the underlying mean shift is a fraction of a percentage point — not something a practitioner should act on. `store_nbr`, `is_regional_holiday`, `is_local_holiday`, and `cluster` correctly show no drift at all, as expected for structural, time-invariant store attributes.

Crucially, this covariate drift had not, at the time of evaluation, translated into any actual performance degradation: the alerting logic (RMSE > 600 or $R^2 < 0.65$) did not fire, since the observed RMSE (331.6) and R^2 (0.9407) were both comfortably within healthy bounds. This is exactly the value proposition of separating data drift from performance drift — the monitoring system surfaced a real, business-relevant early warning (“your promotional and macro environment has shifted materially”) well before it would show up as a measurable accuracy problem.

12.3 Alerting Logic

`log_evaluation` triggers an alert when RMSE exceeds 600 or R^2 falls below 0.65, and separately flags a drift event when any feature's KS test p-value falls below 0.05. Applied to this project's one logged evaluation cycle, `drift_detected` was `True` (correctly, per the discussion above) but `alert_triggered` was `False` — the system distinguishes “something about the input data has changed” from “the model is no longer performing acceptably,” which is the right behavior for an alerting system that should not cry wolf on every distributional wiggle.

12.4 Retraining Strategy

`retrain.py` defines three retraining triggers: a scheduled cadence (every 90 days regardless of performance), a performance-degradation trigger (RMSE increasing by more than 10% relative to the currently deployed model), and a minimum-new-data threshold (at least 30 days of new data before retraining is considered worthwhile). Running the pipeline end-to-end (Milestone 4's “# Model Training” section) re-splits the data chronologically 80/20, re-runs the same 3-fold `TimeSeriesSplit` cross-validation, retrains an XGBoost model with the identical hyperparameters used originally, and then explicitly compares the new model's test RMSE against the currently deployed model's RMSE before deciding whether to promote it.

◆ Reading this project's retraining log correctly

`retraining_config.json` records two historical entries: an initial run (“No previous model version found. Deploying first retrained model,” `deployed = true`) and a second run (“New model degraded RMSE by 0.00%. Keeping current model,” `deployed = false`). Because both runs were executed against the same

historical dataset used to build the original model — there being no genuinely new, post-deployment sales data available within the scope of this project — the second run's RMSE is numerically identical to the first, and the pipeline correctly declines to replace a model with an identical copy of itself. This is best understood as a validated dry run that proves the promotion/rejection logic works correctly, rather than a demonstration of retraining on fresh data; in live operation, this same function would instead be called against newly accumulated sales records, and the comparison would reflect a genuinely different, freshly trained model.

12.5 How This Supports Operational Use

Together, MLflow tracking, KS-based drift detection, threshold-based alerting, and an automated retrain-and-compare pipeline give this project the operational backbone of a real production forecasting service: every model version is reproducible and comparable, performance is checked continuously rather than only at initial training time, and the decision to redeploy a retrained model is made programmatically against an explicit, auditable comparison rather than by assumption.

13. Business Impact

Translating the technical results into operational language:

13.1 Inventory Optimization and Stockout Reduction

At an MAE of roughly 96 units against an average of 358 units per store-family-day, replenishment orders driven by this model should track real demand meaningfully more closely than a naive average- or last-week-based approach, directly reducing both the stockout risk documented in Section 1.3 and the working capital tied up in excess stock — particularly valuable for the perishable top-selling families identified in Section 4.7 (PRODUCE, DAIRY, MEATS all appear in the top-ten families by average sales).

13.2 Demand Planning and Seasonal Readiness

The confirmed December peak (Section 4.3, +33% over the quietest month) and the strong weekend effect (Section 4.2, Sunday +63% over Thursday) give planners concrete, quantified seasonal and weekly targets to build replenishment and labor calendars around, rather than relying on informal intuition about “busy season.”

13.3 Staffing Support

Because the weekly and monthly seasonal patterns are both statistically confirmed and now forecastable at the store-family level, staffing schedules can be built proactively against a specific predicted-demand curve for each store, rather than reactively adjusted after a day is already under- or over-staffed.

13.4 Marketing and Promotional Timing

The measured 7.2× sales lift under active promotion (Section 4.5) is the single largest lever identified anywhere in this analysis. A model that can forecast both baseline demand and promotional uplift by store and family gives merchandising teams a data-driven basis for sizing promotional depth, timing campaigns against naturally slower periods (e.g., counter-balancing the weekday trough identified in Section 4.2), and pre-positioning the replenishment a successful promotion will require.

13.5 Supply Chain Visibility and Manager Decision-Making

The Streamlit “PredictAI” dashboard (Section 12 and Appendix) turns the model into a self-service tool: store or category managers can pull an instant forecast for a specific store/family/date combination, compare predicted performance across stores or families, upload a batch of scenarios for bulk planning, and check the live health of the underlying model — without needing to involve a data scientist for each request.

14. Challenges and Limitations

A credible final report states its own weak points plainly. The following limitations are real, current constraints of this implementation — not reasons to discount the results, but context every stakeholder evaluating the model should have.

14.1 Data Limitations

- No inventory/stock-on-hand data: a zero-sales row cannot be distinguished from “no one wanted this item that day” versus “this item was out of stock or never stocked at this store,” which can bias both EDA interpretation and model learning around the 31.3% zero-sales rows identified in Section 4.1.
- The 2016 Ecuador earthquake (April 16, 2016, magnitude 7.8) falls inside the training window and is known, from the dataset's real-world context (Section 2.1), to have disrupted supermarket demand for weeks afterward, yet is not explicitly flagged as a feature anywhere in the current pipeline — the model can only absorb this shock implicitly, through whatever calendar and store signals happen to correlate with it.
- Ecuador's holiday calendar distinguishes finer categories (Bridge days, Work Day paybacks) beyond the Holiday/Event split this project uses; the coarser split may understate some calendar effects, consistent with the muted, mixed-direction holiday effect found in Section 4.4.

14.2 Feature Availability at Inference Time

As discussed in Section 6.6, lag and rolling-sales features were deliberately left out of the production feature set to keep the live API self-contained, meaning the deployed model does not benefit from recent-sales momentum the way a lag-aware model could. Likewise, `transactions` and `dcoilwtico` are both required inputs to `/predict`; in a fully automated production setting, these would need their own upstream forecasting or scheduled data-refresh process to be available for genuinely future dates, since neither is knowable with certainty in advance (the current API allows them to be supplied by the caller or defaulted to 0, which is a reasonable interim design but not a long-term substitute for a proper upstream feed).

14.3 Seasonal and Covariate Drift

Section 12.2 already documents measurable drift in oil price and promotion intensity between the training and test windows. Because both are real, ongoing macro/business trends rather than one-off anomalies, this drift should be expected to continue evolving after deployment, reinforcing the importance of the scheduled and performance-triggered retraining cadence defined in `retrain.py`.

14.4 External Factors Not Included

Weather, local competitor activity, and broader macroeconomic indicators beyond the oil price (e.g., inflation, consumer confidence) are not part of the current feature set, despite all three being flagged as natural extensions in the original project brief.

14.5 Evaluation Limitations

- The Random Forest hyperparameter search was intentionally narrow (Section 7.2); its reported performance is a solid benchmark rather than a fully tuned ceiling.

- The monitoring and retraining demonstrations in Section 12 were both run against the same historical dataset rather than genuinely new post-deployment data, since none existed within this project's scope — the mechanics are validated, but the modules have not yet been exercised against real drift-driven retraining.
- The project reports MAE/MSE/RMSE/R² on raw sales rather than the original Kaggle competition's own evaluation metric (a normalized, weighted RMSLE computed on log-transformed sales). This is a reasonable and common academic choice, but given the confirmed heavy right skew of the target (skewness 7.36, Section 4.1), a log-scale metric would weight errors on high- and low-volume families more evenly, and is worth adding alongside the current metrics for a more complete picture.

14.6 Production Deployment Caveats

- CORS is currently configured wide open (`allow_origins=["*"]`) in `app.py` — acceptable for prototyping, but it should be restricted to known origins, and authentication/rate-limiting added, before any public-facing production deployment.
- The API and dashboard both assume the model artifact and reference CSVs (`stores.csv`, `holidays_events.csv`) are present on local disk; a production deployment would typically source these from a managed model registry and a proper data warehouse rather than flat files bundled alongside the application code.

14.7 Assumptions Affecting Generalization

The model assumes the statistical relationships learned from 2013–2017 Ecuadorian grocery demand — including the specific magnitude of the promotion effect, the December seasonal peak, and the store-cluster performance hierarchy — remain reasonably stable going forward. The drift analysis in Section 12.2 already shows the promotional and oil-price environment shifting even within this project's own train/test window, which is exactly the kind of assumption the monitoring and retraining loop exists to continuously re-validate rather than take on faith indefinitely.

15. Future Improvements

15.1 Feature Enhancements

- Reintroduce lag and rolling-sales features behind a proper feature store or scheduled batch-refresh process, so the production model can benefit from recent-sales momentum without compromising the API's current simplicity.
- Add an explicit post-earthquake / external-shock flag and a payday-distance feature (Ecuadorian public wages are paid on the 15th and last day of the month), both grounded in this dataset's well-documented real-world context (Section 2.1) but not yet present in the feature set.
- Disaggregate the combined `is_holiday` flag by holiday type rather than a single binary indicator, given the muted, mixed-direction effect found in Section 4.4.
- Incorporate weather data, local competitor activity, and broader macroeconomic indicators (inflation, consumer confidence) alongside the existing oil-price signal.

15.2 Modeling Enhancements

- Widen the Random Forest hyperparameter search (Section 7.2) to test whether a fuller tuning budget narrows its gap to XGBoost.
- Add a feature-importance and/or SHAP analysis for the production XGBoost model, partially restoring the interpretability trade-off noted in Section 11.3 without sacrificing accuracy.
- Evaluate a log-transformed target or an RMSLE-style metric alongside the current MAE/RMSE/R² suite, given the confirmed heavy right skew of sales (Section 14.5).
- Test Prophet or a global deep-learning sequence model (e.g., a temporal fusion transformer) as additional candidates, particularly ones designed to model many related series (store × family) jointly rather than one aggregate series at a time, addressing SARIMA's core structural limitation (Section 7.4).

15.3 MLOps and Deployment Enhancements

- Exercise the monitoring and retraining pipeline against genuinely new, post-deployment data rather than the historical dataset alone (Section 14.5), including validating the scheduled 90-day and performance-degradation retraining triggers under real conditions.
- Harden the FastAPI service for production: restrict CORS to known origins, add authentication and rate limiting, and source the model artifact and reference data from a managed registry/warehouse rather than local files (Section 14.6).
- Extend the Streamlit dashboard with a dedicated drift/monitoring visualization page backed directly by the `monitoring_log.json` history, rather than requiring a notebook or manual report review to see drift trends.
- Automate the full retrain → validate → compare → promote cycle on the defined schedule (currently implemented as callable functions, but not yet wired to a scheduler such as Airflow, cron, or a cloud-native pipeline trigger).

16. Conclusion

This project set out to build a full, end-to-end sales forecasting capability for a large Ecuadorian grocery retailer — not just a notebook that fits a model, but a complete pipeline from raw, multi-source data through cleaning, exploratory and time-series analysis, feature engineering, rigorous time-aware model validation, and into an operational MLOps layer with real monitoring and retraining logic. Across four candidate models evaluated under a strictly chronological validation scheme, an XGBoost Regressor emerged as the clear winner, explaining 94.1% of the variance in store-family-day sales with a mean absolute error of roughly 96 units — a result grounded in, and consistent with, the strong promotional, weekly, and seasonal patterns uncovered during exploratory analysis.

The final model is not left as a static artifact: it is served through a documented FastAPI service, exposed to business users through a purpose-built Streamlit dashboard, tracked through MLflow, and wrapped in a drift-detection and retraining loop that has already surfaced a real, business-relevant signal — meaningful shifts in oil price and promotional intensity between the training and test periods — well ahead of any measurable drop in accuracy. Where the current implementation falls short of a fully mature production system — in feature availability at inference time, in the breadth of external signals modeled, in production hardening, and in having yet to be exercised against genuinely new data — those gaps are documented plainly in Sections 14 and 15 as the concrete, prioritized next steps for continuing this work.

Taken together, the modeling result and the operational scaffolding around it give Corporación Favorita — or any retailer facing the same forecasting challenge — a credible, extensible foundation for turning historical sales data into sharper inventory, staffing, and promotional decisions.

17. References

- Dataset: Kaggle, “Store Sales – Time Series Forecasting,” <https://www.kaggle.com/competitions/store-sales-time-series-forecasting> — sales, store, oil, holiday, and transaction data from Corporación Favorita, an Ecuadorian grocery retailer.
- pandas, NumPy — data manipulation and numerical computing.
- Matplotlib, Seaborn, Plotly (plotly.express, plotly.graph_objects) — exploratory, statistical, and interactive visualization.
- statsmodels (seasonal_decompose, adfuller, SARIMAX) — time-series decomposition, stationarity testing, and the SARIMA benchmark model.
- scikit-learn (LinearRegression, RandomForestRegressor, ColumnTransformer, OneHotEncoder, TimeSeriesSplit, RandomizedSearchCV) — preprocessing pipeline, baseline and ensemble modeling, and time-aware validation.
- XGBoost (XGBRegressor) — final production gradient-boosting model.
- MLflow — experiment tracking, metric/parameter/artifact logging.
- FastAPI, Pydantic, Uvicorn — production REST API for real-time and batch forecasting.
- Streamlit — the “PredictAI” business-intelligence dashboard.
- SciPy (stats.ks_2samp) — Kolmogorov-Smirnov drift detection.
- joblib — model artifact serialization.

18. Appendix

A. Deployed System Components

File	Role
app.py	FastAPI service: /, /health, /model/info, POST /predict, POST /predict/batch, GET /predict/range, GET /predict/range/auto
feature_engineering.py	Shared feature-construction module (holiday-flag logic, store lookup) used identically by the API and dashboard
dashboard.py	“PredictAI” Streamlit dashboard: Home/KPIs, Single Prediction, Batch Prediction, Forecast Analysis, Analytics, Model Comparison, Live Monitoring
monitoring.py	evaluate_model, detect_data_drift (KS-test), log_evaluation (threshold alerting), generate_monitoring_log_to_mlflow
retrain.py	should_retrain (scheduled / performance / min-new-data triggers), retrain_model (full retrain → complete promote pipeline)
model_metrics.json / monitoring_log.json / retraining_config.json	Persisted metrics, monitoring history, and retraining/version history

Table 9

B. PredictAI Dashboard — Page Summary

Page	Purpose
Home	KPI cards (model status, algorithm, 54 stores, 33 families) and a quick single-click forecast widget
Single Prediction	On-demand forecast for one store / family / date combination, auto-filling store metadata
Batch Prediction	CSV upload for bulk forecasting with a downloadable results file
Forecast Analysis	Trend visualization of forecasted demand over a selected date range
Business Analytics	Side-by-side store and product-family comparison charts
Model Comparison	RMSE / R ² bar charts across all four models, with a highlighted “best model” summary card
Live Monitoring	API health check, model/metrics/file-system status, and system (Python/pandas/NumPy) version info

Table 10

C. Final XGBoost Hyperparameters

Parameter	Value
n_estimators	300
learning_rate	0.05
max_depth	8
subsample	0.8
colsample_bytree	0.8
objective	reg:squarederror
random_state	42

Table 11

These exact hyperparameters are shared between the exploratory notebook and the standalone `retrain.py` script, ensuring any future automated retraining cycle reproduces the same model configuration validated in this report.

D. SARIMA Model Diagnostics

SARIMAX(1,1,1)(1,1,1,7) on aggregated daily sales — AIC = 34,232.4, BIC = 34,258.4. Coefficient significance: ar.L1 = 0.4615 (p < 0.001), ma.L1 = -0.8729 (p < 0.001), ar.S.L7 = -0.0235 (p = 0.652, not significant), ma.S.L7 = -0.9276 (p < 0.001). The Ljung-Box test on residuals (p = 0.88) shows no remaining significant autocorrelation, indicating the model has captured the series' autocorrelation structure reasonably well; however, the Jarque-Bera test strongly rejects residual normality (skew = -0.88, kurtosis = 29.12), reflecting the heavy-tailed, spike-driven nature of daily retail sales that a linear Gaussian model can capture only partially — consistent with SARIMA's comparatively weaker R² in Section 10.

E. Reproducibility Notes

requirements.txt pins minimum versions for the full stack: fastapi≥ 0.104.0, uvicorn≥ 0.24.0, streamlit≥ 1.28.0, plotly≥ 5.18.0, pandas≥ 2.0.0, numpy≥ 1.24.0, scikit-learn≥ 1.3.0, xgboost≥ 2.0.0, joblib≥ 1.3.0, mlflow≥ 2.8.0, scipy≥ 1.11.0, and pydantic≥ 2.0.0, covering every library used across the notebook, API, dashboard, monitoring, and retraining code.